



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

DIPARTIMENTO DI INFORMATICA – SCIENZE E INGEGNERIA - DISI

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

.....

.....

DIGITAL TWIN CON COMUNICAZIONI DETERMINISTICHE BASATE SU TSN E OPC UA FX

Tesi di laurea magistrale

Relatore

Prof. Paolo Bellavista

Presentata da

Christian Candela

Correlatore

Ing. Riccardo Venanzi

Sessione marzo 2026

Anno Accademico 2024/2025

Introduzione

L'evoluzione verso il paradigma dell'industria 5.0, focalizzata non solo sull'efficienza ma anche sulla resilienza dei sistemi, unita all'introduzione di tecnologie avanzate come il Digital Twin richiede una forte convergenza tra le reti informatiche (IT) e quelle operative (OT). Per permettere a un gemello digitale di operare in tempo reale su processi fisici è necessario garantire comunicazioni deterministiche. Le reti Ethernet tradizionali sono però basate su approcci Best Effort che introducono latenze e jitter variabili ed imprevedibili, rendendo questa tecnologia non adatta al traffico di controllo critico.

Questo elaborato presenta la progettazione e l'implementazione di ASTRID (Advanced Support for Time-sensitive Reliable Industrial Digital-twins), un'architettura di rete che supera i limiti dell'Ethernet standard integrando i protocolli Time Sensitive Networking (TSN) e OPC UA Field eXchange (FX). Il framework è stato sviluppato in Python realizzando un'architettura ibrida: un canale Publisher/Subscriber su base UDP per la trasmissione Real-Time e un canale Client/Server su base TCP per la configurazione. Al fine di minimizzare l'uso della banda e i tempi di parsing, è stata adottata la codifica binaria ottimizzata UADP. La topologia hardware messa a disposizione dalla linea pilota del Competence Center Bi-Rex include un PC industriale Advantech (architettura x86) usato come nodo Publisher, uno switch Relyum TSN Bridge e un Nvidia Jetson Orin (arm) che opera come Digital Twin (Subscriber). Il determinismo temporale è stato ottenuto sincronizzando a livello hardware i dispositivi tramite il protocollo IEEE 802.1AS (gPTP) e implementando il meccanismo di schedulazione delle code Time Aware Shaper (TAS) IEEE 802.1Qbv. Quest'ultimo ha permesso di confinare il traffico critico sulla VLAN 100 con priorità PCP 6. ASTRID, inoltre, prevede una pipeline di monitoraggio basata su InfluxDB e Grafana.

La validazione di ASTRID è stata realizzata tramite una serie di test eseguiti su rete satura, grazie all'introduzione di traffico di disturbo fino a 900 Mbps generato con iperf3. I risultati ottenuti mostrano la netta superiorità della rete TSN. Nello scenario Ethernet standard si è assistito ad un forte degrado delle prestazioni, con una latenza di 1.5 ms e un jitter medio che ha raggiunto gli 11.8 ms. L'architettura TSN ha invece mantenuto una latenza media di 0.20 ms e un jitter stabile intorno ai 0.42 ms. Il traffico

critico è quindi stato ben isolato rispetto al flusso di dati di disturbo. Un'analisi approfondita dell'overhead ha evidenziato che in un sistema TSN il collo di bottiglia si sposta dalla rete fisica allo stack software. L'elaborazione software ha infatti registrato ritardi di 1.67 ms a fronte di solo 0.79 ms di latenza hardware. Questo test dimostra come l'uso di linguaggi interpretati in user space introduca latenza che può essere abbattuta utilizzando linguaggi più a basso livello e sistemi operativi RealTime.

Indice

Introduzione	1
Capitolo 1	5
1.1 Industria 4.0 e 5.0.....	5
1.2 Cloud Continuum.....	7
1.3 Digital Twin	9
1.3.1 Classificazione per oggetto	10
1.3.2 Classificazione per flusso di dati	10
1.4 Dal Best Effort al determinismo	11
1.4.1 Hard Real-Time vs Soft Real-Time	12
1.4.2 L'indeterminismo di Ethernet	12
1.4.3 Time Sensitive Networking (TSN)	13
1.5 Obiettivi della tesi	14
Capitolo 2: Stato dell'arte e tecnologie.....	16
2.1 Il protocollo OPC UA: dal Client/Server al Field eXchange.....	16
2.1.1 Modello Client/Server vs PubSub.....	16
2.1.2 Codifica UADP (Unified Architecture Datagram Packet).....	18
2.2 Virtualizzazione e Containerizzazione nel Cloud Continuum.....	20
2.2.1 Dalle virtual machine ai container	20
2.2.2 Docker.....	21
2.2.3 Orchestrazione e Kubernetes	22
2.3 Time Sensitive Networking (TSN)	22
Capitolo 3: Architettura del Sistema ASTRID	27
3.1 Architettura generale del sistema.....	27
3.1.1 Topologia fisica.....	27
3.1.2 Topologia Logica	28
3.1.3 Strategie di deployment	29
3.2 Configurazione dell'infrastruttura di rete	29
3.2.1 Configurazione delle interfacce virtuale (VLAN)	29
3.2.2 Configurazione dello switch	30
3.2.3 Prioritizzazione del traffico.....	31
3.3 Sincronizzazione temporale (IEEE 802.1AS).....	33
3.3.1 Gerarchia PTP	33
3.3.2 Implementazione con LinuxPTP.....	33
3.4 Implementazione software	35

3.4.1 Publisher: architettura ibrida.....	36
3.4.2 Subscriber: Decodifica e persistenza	39
3.5 Archiviazione dati e dashboard.....	42
3.5.1 Persistenza dei dati: InfluxDB	43
3.5.2 Dashboard Grafana	44
Capitolo 4: Test e analisi dei risultati.....	46
4.1 Setup degli esperimenti.....	46
4.1.1 Metodologie di stress della rete	46
4.1.2 Metodologie di test.....	47
4.2 Analisi della latenza	48
4.2.1 Acquisizione dei dati.....	48
4.2.2 Calcolo della latenza	48
4.2.3 Analisi dei risultati	49
4.3 Analisi del jitter.....	54
4.3.1 Metodologia di misurazione.....	54
4.3.2 Analisi dei risultati	55
4.4 Analisi dell'overhead dello stack software	58
4.4.1 Metodologie di misurazione.....	58
4.4.2 Analisi dei risultati	59
4.5 Monitoraggio e Visualizzazione Real-Time tramite Grafana	61
4.5.1 Configurazione della dashboard.....	62
Conclusioni e Sviluppi Futuri	65
Sintesi delle attività e obiettivi raggiunti	65
Analisi dei limiti implementativi	65
Sviluppi futuri	66
Indice delle figure	67
Bibliografia	68

Capitolo 1

1.1 Industria 4.0 e 5.0

Il settore industriale è segnato da continui adattamenti volti a rispondere alle esigenze del mercato e alle innovazioni tecnologiche. Tale processo evolutivo è iniziato con la prima rivoluzione industriale nel '700. Con l'introduzione della macchina a vapore si è definito il paradigma dell'industria 1.0, con conseguente aumento della velocità e della potenza di produzione. Nell'800, l'arrivo dell'elettricità ha definito il paradigma dell'industria 2.0, segnato dall'introduzione della catena di montaggio e dalla produzione di massa. A partire dagli anni 70 abbiamo invece assistito all'incontro tra informatica ed elettronica, in quella che è l'industria 3.0. Sono apparsi i primi controllori logici programmabili (PLC) e le prime linee di produzione automatizzate da robot.^[1]

Diretta evoluzione di tale incontro è l'industria 4.0, basata sui dati e sull'interconnessione. Nei processi produttivi diventano cruciali i dispositivi IoT, l'intelligenza artificiale, i Big Data e il Cloud Computing. Grazie ai sensori IoT è possibile monitorare in Real-Time lo stato dei macchinari permettendo la realizzazione della manutenzione predittiva e una rapida riconfigurazione. L'obiettivo è quello di poter intervenire tempestivamente sulla produzione in caso di malfunzionamenti, in modo da ridurre tempi di inattività, ma anche facilitare la modifica della linea produttiva nel caso in cui il mercato lo richiedesse. I dati raccolti non si fermano però all'interno della fabbrica ma si estendono anche all'esterno. Opinioni dei clienti, andamento del mercato sono solo alcune delle informazioni che possono essere usate per migliorare l'efficienza. Vista la mole di dati immagazzinati e la necessità di analizzarli in tempo reale il supporto dell'IA e del machine learning è fondamentale per migliorare il processo decisionale. Un altro aspetto cruciale dell'industria 4.0 è l'integrazione dei macchinari preesistenti (brownfield) tramite IoT gateway. Questi ultimi permettono di tradurre protocolli di campo proprietari ed obsoleti in standard moderni per la trasmissione dati verso il cloud.^[2]

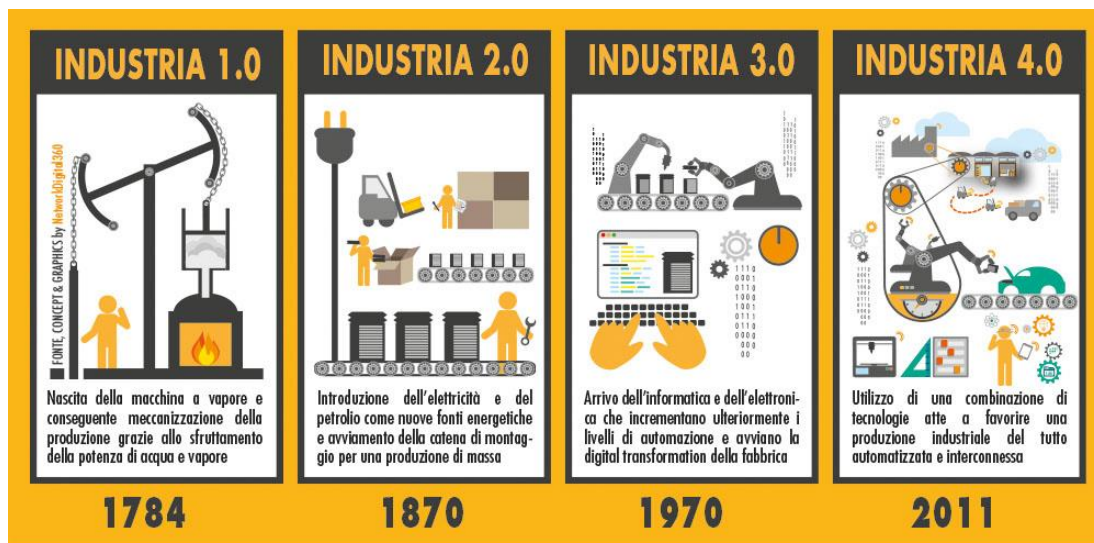


Figura 1 - Evoluzione industriale¹

Con l'industria 5.0 si ha invece un cambio di paradigma che non si ferma alla semplice evoluzione tecnologica, ma che diventa una vera e propria evoluzione culturale. Oltre all'efficienza produttiva, l'industria 5.0 pone l'accento su tre pilastri: la centralità dell'uomo, la sostenibilità e la resilienza. La tecnologia deve adattarsi all'operatore, guidandolo e garantendo la sua sicurezza. I processi devono ridurre l'impatto ambientale e soprattutto devono diventare il più resilienti possibili. Devono garantire continuità operativa anche durante crisi globali come conflitti geopolitici o pandemie e non solo in caso di interruzioni di rete. Le architetture devono quindi diventare flessibili e robuste non potendo, ad esempio, dipendere da un unico cloud centralizzato.^[3]

I requisiti di resilienza dell'industria 5.0 e la necessità della comunicazione Real-Time sottolineano i limiti del modello cloud centrico dell'industria 4.0. Il determinismo temporale non può tollerare la latenza dovuta all'invio dei dati dal livello campo al cloud e viceversa. Per rispondere a questa esigenza, le architetture stanno evolvendo verso un modello maggiormente distribuito che unisce il campo (OT) e il cloud (IT). In questo contesto emerge il concetto di Cloud Continuum che estende la capacità di

¹ Foto: <https://www.digital4.biz/executive/industria-40-storia-significato-ed-evoluzioni-tecnologiche-a-vantaggio-del-business/>

calcolo del cloud ai margini della rete (Edge) permettendo di eseguire elaborazioni anche in nodi più vicini alle macchine.

1.2 Cloud Continuum

Un approccio puramente centralizzato non è più sostenibile per soddisfare i requisiti stringenti del nuovo paradigma dell'Industria 5.0. Il Cloud Computing offre una scalabilità e una potenza di calcolo apparentemente infinite ma non garantisce bassa latenza e risposta in tempo reale. La necessità di un cambio di paradigma non è dettata solo dai requisiti temporali, ma anche dai vincoli di banda. Nell'Industria 4.0, la mole di dati grezzi generata dai sensori (Big Data) ha reso inefficiente il trasferimento massivo verso il Cloud centrale, un fenomeno noto come Data Gravity. Inoltre, la sensibilità dei dati industriali impone spesso che le informazioni critiche non lascino il perimetro della fabbrica per motivi di sicurezza.

Per rispondere a queste esigenze è stata definita una nuova tipologia di architettura: il Cloud Continuum.

Il termine indica un'estensione del cloud tradizionale a cui sono aggiunte molteplici entità più vicine a dove i dati vengono prodotti, costruendo quello che definiamo Edge o Fog cloud. Non si tratta di un'architettura monolitica, ma di un ecosistema fluido che integra due visioni differenti: la distribuzione delle risorse fisiche, un insieme di sistemi hardware che va dai sensori IoT ai data center del Cloud, e l'estensione del processing, ovvero la capacità di eseguire carichi di lavoro ovunque lungo la rete.

All'interno del Cloud Continuum le risorse sono distribuite con una specifica gerarchia e a ciascun livello è assegnato un ruolo:

- Cloud Level: Rappresenta le risorse "illimitate" del cloud. È il livello adatto per l'analisi storica, l'addestramento dei modelli di Intelligenza Artificiale e lo storage a lungo termine.
- Fog Level: Agisce come livello intermedio che garantisce flessibilità e adattamento. I nodi Fog non si limitano al trasferimento dati, ma aggregano le

informazioni provenienti dalla periferia, filtrano il traffico di rete per ridurre l'occupazione di banda e mediano tra i protocolli di campo e i servizi Cloud.

- Edge Level: Man mano che ci si sposta verso la periferia il focus passa dalla scalabilità all'affidabilità. A questo livello l'infrastruttura deve garantire l'elaborazione locale dei file e la comunicazione a bassa latenza con capacità deterministiche.

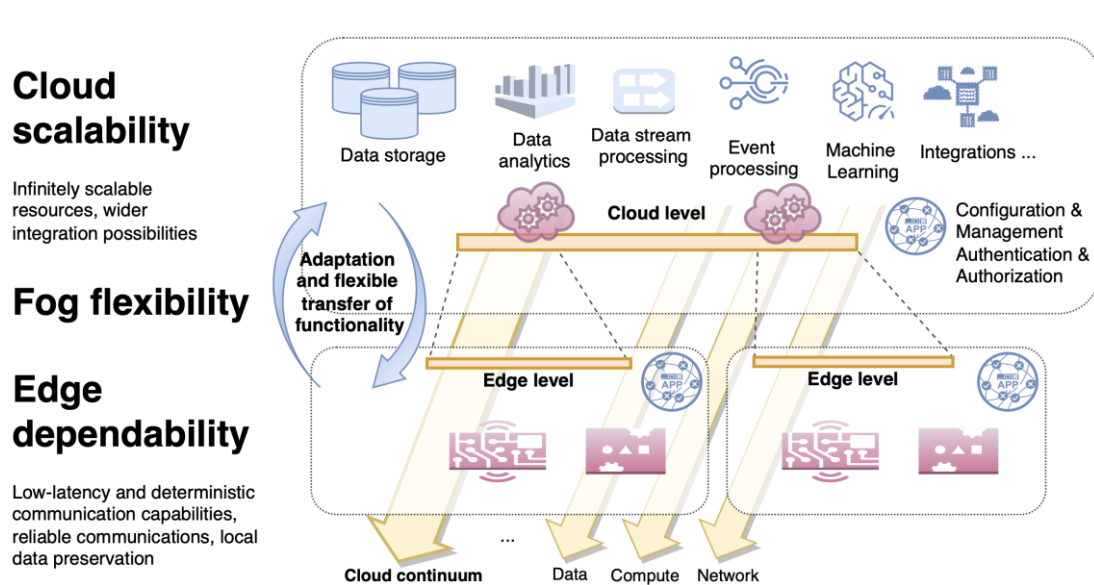


Figura 2 - Cloud Continuum ²

Altro elemento determinante è l'utilizzo della virtualizzazione, in particolare la containerizzazione realizzata, ad esempio, con Docker e Kubernetes. Questa tecnologia disaccoppia il software dall'hardware permettendo di spostare il carico computazionale dove serve. Un microservizio che deve ricevere una risposta immediata da un sensore non comunica più con il cloud (latenza variabile introdotta dalla connessione di rete) ma direttamente con il dispositivo IoT. Se ovviamente questo controllo è considerato critico è necessario che la rete sottostante supporti garanzie temporali superando il comportamento Best Effort delle reti tradizionali.^[4]

Il Cloud Continuum è uno strato necessario per le applicazioni industriali moderne. A beneficiare maggiormente della capacità di calcolo dell'Edge e della bassa latenza è il

² Foto: https://www.researchgate.net/publication/366277900_Cloud_Continuum_The_Definition

Digital Twin che da strumento di monitoraggio diventa un vero e proprio controllore attivo. Interagendo con il Digital Twin, diventa possibile agire sul processo produttivo fisico.

1.3 Digital Twin

Una delle tecnologie più promettenti nell'ecosistema dell'industria 4.0/5.0 e supportata dall'infrastruttura del Cloud Continuum è sicuramente il Digital Twin. Esso può essere definito come una rappresentazione virtuale dinamica di un oggetto o di un sistema in tutto il suo ciclo di vita. Viene aggiornato tramite dati in tempo reale e sfrutta la simulazione e il machine learning per supportare il processo decisionale aziendale. Il concetto fu introdotto per la prima volta nel 2002 da Michael Grieves presso l'Università del Michigan, inizialmente come modello per la gestione del ciclo di vita del prodotto (PLM).

Conceptual Ideal for PLM

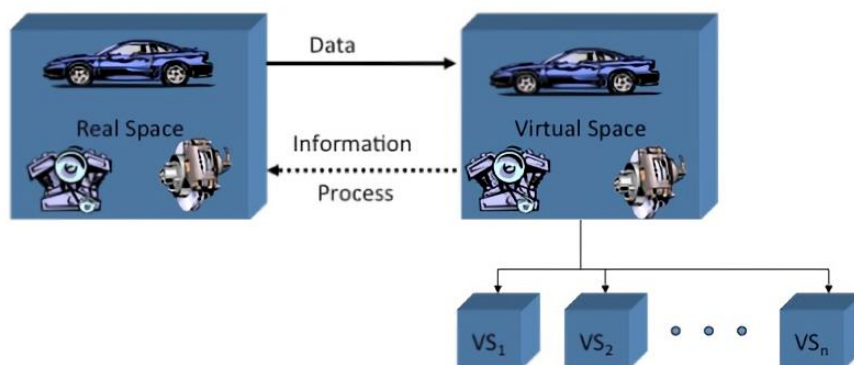


Figura 3 - Ciclo di vita del prodotto³

A differenza di una simulazione statica, il gemello digitale è dinamico ed è costruito grazie a informazioni acquisite costantemente attraverso sensori e dispositivi IoT

³ Foto: <https://www.ai4business.it/intelligenza-artificiale/digital-twin-cose-e-come-funziona-il-modello-del-gemello-digitale/>

installati sull'asset fisico. I dati vengono trasmessi al sistema di elaborazione (Edge o Cloud) permettendo di monitorare lo stato, prevedere guasti e prestazioni senza interrompere l'operatività della parte fisica.^[7]

1.3.1 Classificazione per oggetto

In base all'ambito di applicazione è possibile distinguere tre categorie: ^[4]

- **Product Digital Twin:** Replicazione di un prodotto o di un componente fisico. Viene utilizzato fin dalla fase di design per simulare il comportamento di un prodotto e ottimizzare le prestazioni prima che venga realizzato.
- **Production Digital Twin (di processo):** Replicazione dell'intero processo di produzione per analizzare l'efficienza delle linee, prevedere colli di bottiglia e ottimizzare la manutenzione.
- **Performance Digital Twin (di sistema):** Replicazioni di interi sistemi complessi in cui interagiscono diversi asset come, ad esempio, reti di trasporti o Smart City. Integra dati provenienti da diversi gemelli di prodotto o di processo per fornire una visione su larga scala.

1.3.2 Classificazione per flusso di dati

È possibile classificare i Digital Twin in base al livello di automazione nel flusso dei dati tra la parte fisica e quella digitale:^[6]⁴

⁴ Foto: <https://www.skilla.com/blog/digital-twin-la-convergenza-tra-virtuale-e-reale/>

- **Digital Model:** Rappresentazione digitale di un oggetto in cui non c'è nessuno scambio automatico di dati. Il trasferimento delle informazioni avviene solo manualmente.
- **Digital Shadow:** Esiste un flusso automatico di dati unidirezionale dal sistema fisico a quello digitale. Un cambiamento di stato nell'oggetto fisico si riflette automaticamente nel modello digitale (monitoraggio), ma non viceversa.
- **Digital Twin:** Il flusso di dati è bidirezionale e automatico. Non solo il modello virtuale si aggiorna in tempo reale, ma è in grado di inviare comandi di controllo per influenzare e modificare lo stato del sistema fisico.

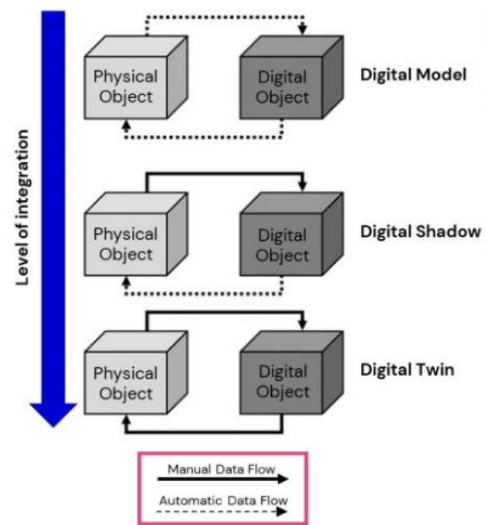


Figura 4 - Classificazione su flusso dati

Il passaggio delicato si ha proprio tra il Digital Shadow e il Digital Twin. Affinché il gemello virtuale possa agire sul sistema reale è necessario che ci sia una sincronizzazione tra mondo virtuale e fisico. Se la rete introduce latenza, il comando inviato dal Digital Twin potrebbe arrivare in ritardo rendendo il controllo inefficiente e in alcuni casi anche pericoloso. Questa necessità di determinismo è il motivo per cui è necessario adottare standard di rete evoluti come Time Sensitive Networking (TSN).

1.4 Dal Best Effort al determinismo

L'utilizzo di un Digital Twin capace di inviare comandi di controllo all'impianto fisico (field level) impone requisiti di rete che non sono soddisfatti dalla semplice larghezza di banda. I dati devono viaggiare con tempistiche certe e replicabili. Per questo motivo è necessario utilizzare nuovi standard di rete.

1.4.1 Hard Real-Time vs Soft Real-Time

Prima di analizzare i limiti della rete Ethernet è necessario definire cosa si intende per sistema Real-Time. Un sistema in tempo reale non è necessariamente veloce, ma è sicuramente predicibile. La sua correttezza non dipende solo dal risultato ottenuto ma anche dall'istante in cui è stato prodotto. Questi tipi di sistemi vengono solitamente classificati in due categorie principali in base ai vincoli temporali.

Nei sistemi Hard Real-Time il mancato rispetto di una deadline può portare a conseguenze catastrofiche. Un esempio possono essere i sistemi di controllo di un'automobile (ABS o Airbag). La progettazione di questi apparati richiede che il tempo di risposta sia inferiore alla deadline anche nel caso peggiore possibile definito come WCET (Worst Case Execution Time).

Nei sistemi Soft Real-Time l'obiettivo è quello di massimizzare la Qualità di Servizio. Il mancato rispetto di una deadline non porta a guasti catastrofici ma a un degrado delle prestazioni. Esempi tipici sono le applicazioni di monitoraggio o la visualizzazione di dati provenienti da un Digital Twin.^[15]

1.4.2 L'indeterminismo di Ethernet

Lo standard Ethernet (IEEE 802.3) è stato concepito per applicazioni Best Effort, dove l'obiettivo è quello di massimizzare il volume di dati trasmesso. Non garantisce in alcun modo la puntualità di consegna del singolo pacchetto.

L'impossibilità di Ethernet di garantire il Real-Time deriva dal modo in cui viene gestito l'accesso al mezzo. Nelle prime versioni (Shared Ethernet), l'uso del protocollo CSMA/CD rende il tempo di trasmissione casuale a causa delle collisioni fisiche tra i messaggi. Ogni nodo ascolta il canale prima di trasmettere, ma a causa del tempo di propagazione del segnale fisico nel cavo esiste una finestra temporale in cui due nodi rilevavano il canale come libero. Iniziano quindi a trasmettere contemporaneamente causando una collisione. La trasmissione viene pertanto interrotta e i nodi coinvolti devono ritentare l'invio. Il problema del determinismo risiede nell'algoritmo di

risoluzione della contesa (Binary Exponential Backoff). Per evitare una nuova collisione immediata, ciascun nodo attende un intervallo di tempo casuale estratto da una finestra che raddoppia a ogni fallimento consecutivo. Di conseguenza, il tempo necessario per accedere alla rete non è fisso, ma è una variabile aleatoria che dipende dal carico del traffico istantaneo.

L'introduzione degli switch ha eliminato la collisione sul mezzo fisico visto che le connessioni sono diventate punto a punto, ma non ha risolto il problema del

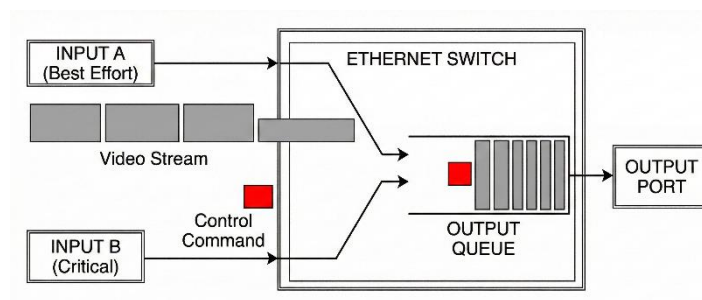


Figura 5 - Meccanismo di accodamento

determinismo. Il collo di bottiglia in questa rete diventa la coda di uscita dello switch. Quando più flussi di dati convergono contemporaneamente verso la stessa porta, i pacchetti

devono essere accodati. Questo accodamento introduce un ritardo variabile che dipende dal carico della rete. Inoltre, anche se un pacchetto ha una priorità alta ma arriva allo switch un istante dopo che è iniziata la trasmissione di un pacchetto dati molto lungo, deve attendere l'intero invio di quest'ultimo.^[8]

1.4.3 Time Sensitive Networking (TSN)

TSN rappresenta l'evoluzione necessaria per permettere la convergenza tra le reti informatiche (IT) e quelle operative (OT). L'obiettivo di TSN non è quello di creare una rete separata, ma di permettere al traffico eterogeneo di convivere sullo stesso cavo. È quindi possibile trasmettere dati ad alta priorità come i comandi del Digital Twin insieme a dati meno critici, come streaming video o log, senza interferenze. Per garantire il determinismo TSN si basa su due pilastri: la sincronizzazione temporale tra i nodi e un sistema di schedulazione temporale del traffico. La rete crea delle finestre temporali riservate: il traffico Best Effort viene bloccato preventivamente per lasciare il cavo libero al traffico critico nel momento in cui deve transitare. In questo

modo i dati di controllo hanno una latenza garantita, rendendo la comunicazione Ethernet deterministica.^[9]

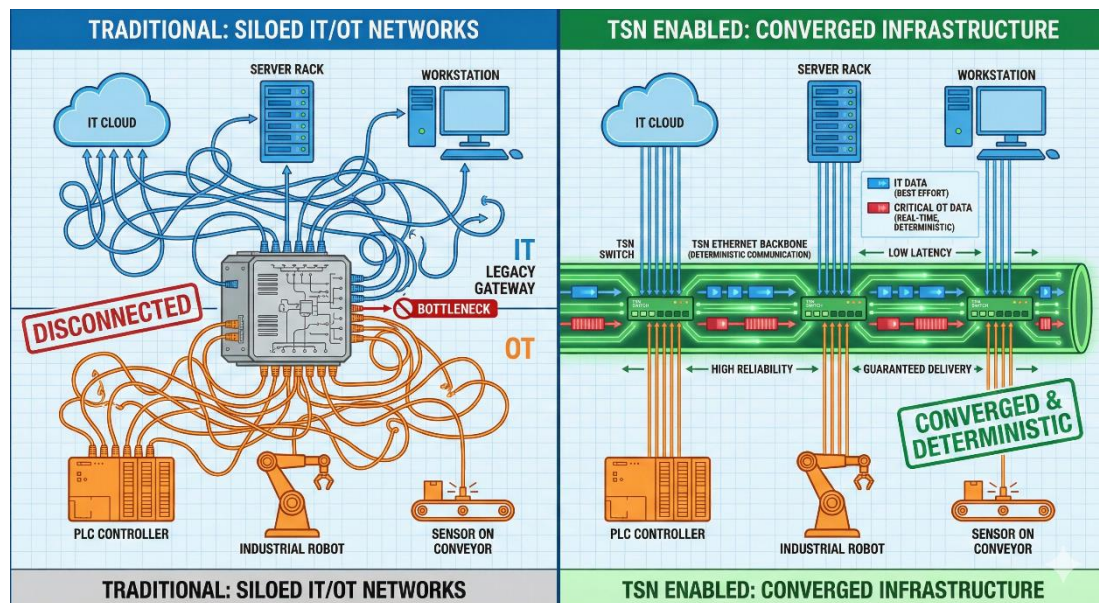


Figura 6 - Traditional IT/OT VS Converged Infrastructure

1.5 Obiettivi della tesi

Gli obiettivi di questo elaborato possono essere suddivisi in quattro macroaree:

1. Sviluppo di uno stack OPC UA FX in Python: Progettazione e implementazione di ASTRID, composto da un modello Publisher/Subscriber basato sullo standard OPC UA Field eXchange (mapping UADP su UDP) e un canale Client/Server. La sfida principale consiste nell'utilizzare la libreria opcua-asyncio per gestire la trasmissione e la ricezione ad alta frequenza.
2. Configurazione dei meccanismi di prioritizzazione del traffico: Implementare una strategia di classificazione dei pacchetti. L'obiettivo è intercettare il traffico UDP generato dal Publisher e applicare il tagging corretto secondo lo standard IEEE 802.1Q (VLAN PCP 6). Nello specifico, si utilizzeranno strumenti come iptables per marcare i pacchetti critici con priorità elevata, garantendone l'accesso alle code riservate degli switch TSN e prevenendo i ritardi tipici del traffico Best Effort.

3. Realizzazione di una pipeline di monitoraggio: Sviluppare un sistema di acquisizione dati per la storicizzazione e l'analisi. Si utilizzerà un Time Series Database (InfluxDB) per la persistenza dei dati e la piattaforma Grafana per la loro visualizzazione.
4. Test ed esperimenti in ambiente industriale: Validare le prestazioni di ASTRID attraverso una serie di test presso la linea pilota di Bi-Rex. L'obiettivo finale è confrontare le prestazioni di una rete Ethernet standard rispetto alla soluzione TSN implementata.

Capitolo 2: Stato dell'arte e tecnologie

2.1 Il protocollo OPC UA: dal Client/Server al Field eXchange

L'OPC Unified Architecture (OPC UA) è lo standard di riferimento per l'interoperabilità nell'automazione industriale. Per estendere la sua applicazione dal livello di controllo (ad esempio SCADA) al livello di campo (Field Level) è necessario passare da una comunicazione Client/Server a una Publisher/Subscriber che supporti la comunicazione Real-Time.

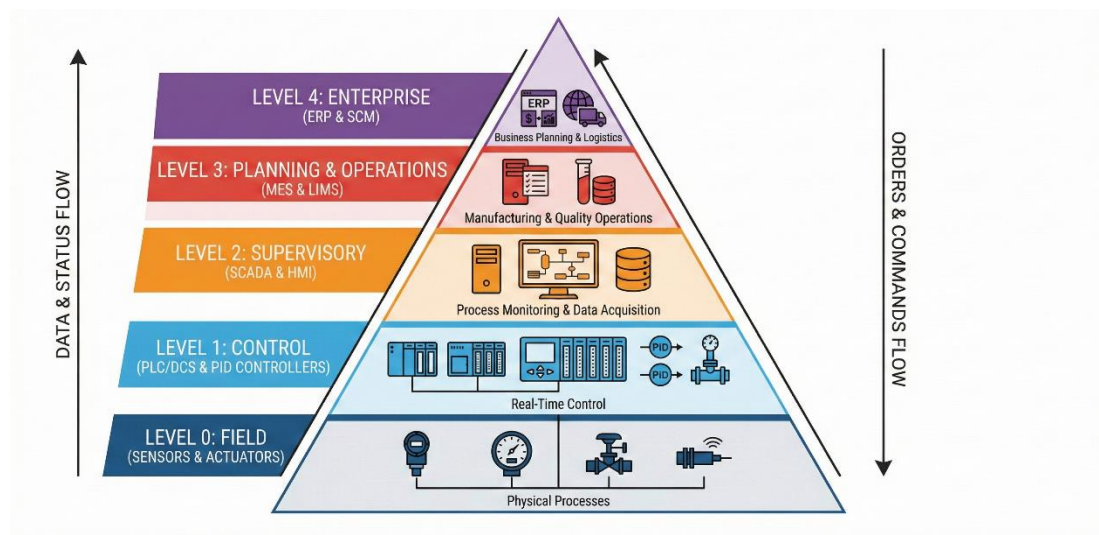


Figura 7 - Piramide dell'automazione

2.1.1 Modello Client/Server vs PubSub

L'architettura classica di OPC UA si basa sul modello Client/Server su TCP. Il client apre una connessione punto a punto con un server, invia una richiesta e attende una risposta. Questa struttura è ideale per operazioni di configurazione o di lettura dei dati ma presenta limiti in scenari many-to-many o Real-Time. Il client deve conoscere la posizione del server, inoltre, inviare dati a più destinatari richiede di aprire molteplici connessioni e la connessione TCP aggiunge un overhead che può saturare la rete in

caso di invio costante di dati a distanza di pochi millisecondi a causa dei messaggi di ACK.

Per superare questi limiti, la specifica OPC UA ha introdotto il modello Publish/Subscribe. Il pubsub disaccoppia le entità in:

- **Publisher:** Entità che produce i dati e li pubblica sulla rete senza sapere chi li riceverà. L'invio dei dati avviene attivamente senza attendere alcuna richiesta esterna.

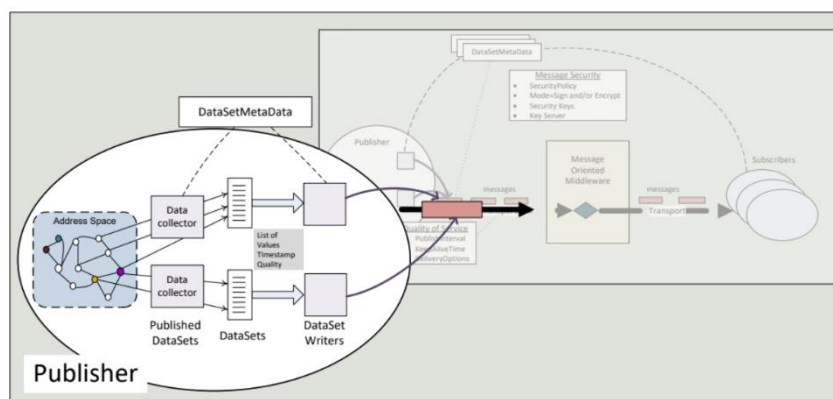


Figura 8 – Publisher

- **Subscriber:** Entità interessata ai dati. Riceve i messaggi e filtra quelli di interesse. La comunicazione avviene senza stabilire una connessione diretta con la sorgente.

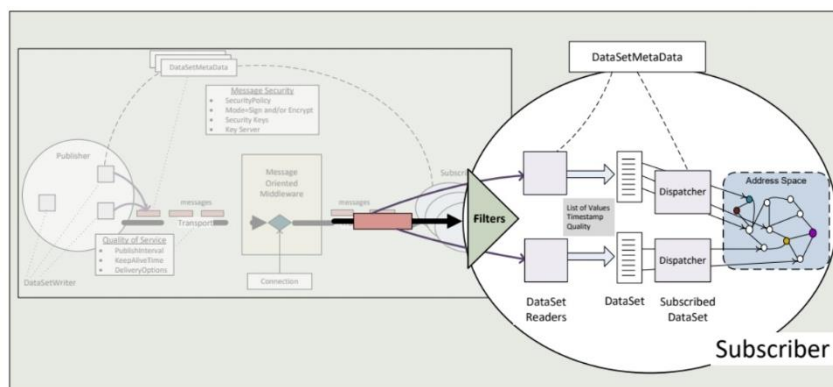


Figura 9 - Subscriber

Questa divisione permette una comunicazione multicast efficiente e una forte scalabilità.

Una nota doverosa deve essere fatta sui protocolli IoT come MQTT. Nonostante utilizzino il paradigma PubSub, si affidano tipicamente ad un broker centrale e al trasporto TCP. Questo aggiunge latenze non deterministiche che non li rendono adatti a una comunicazione Real-Time. Al contrario, OPC UA FX utilizza un approccio Broker-less su UDP multicast. Il publisher, quindi, non invia il pacchetto ad un indirizzo IP specifico ma ad un indirizzo IP multicast che indica un gruppo. Quando il pacchetto arriva allo switch di rete viene inoltrato a tutte le porte dove ci sono dispositivi iscritti ad uno specifico gruppo. Non c'è nessun server o software in mezzo che si occupa di processare il dato. L'overhead si riduce perché non è presente l'handshake del protocollo TCP e il Publisher invia un solo pacchetto anche se è necessario raggiungere molti Subscriber, questo diminuisce anche il carico sulla CPU. Sebbene lo standard OPC UA FX prediliga l'uso del Multicast per l'efficienza one-to-many, il protocollo supporta nativamente anche la trasmissione unicast (Point-to-Point). Quest'ultima modalità è funzionalmente equivalente in scenari con un singolo Subscriber e permette di testare le funzionalità Real-Time riducendo la complessità di configurazione degli apparati di rete (es. IGMP Snooping).^[10]

2.1.2 Codifica UADP (Unified Architecture Datagram Packet)

Il traffico UA FX richiede un certo grado di efficienza e compattezza, per questo motivo non viene utilizzata la codifica JSON ma una codifica ottimizzata chiamata UADP. Questa è progettata specificamente per trasportare datagrammi (come UDP) e per dispositivi con risorse limitate (microcontrollori embedded). I messaggi UADP sono così composti da:

- NetworkMessage: È il contenitore principale inviato sulla rete (il payload del pacchetto UDP). Contiene un Header, con al suo interno ID del Publisher e timestamp, e il Payload.

- DataSetMessage: All'interno del payload del NetworkMessage possono essere aggregati uno o più DataSetMessage. Ognuno di questi rappresenta un aggiornamento di un gruppo di variabili (DataSet).
- DataSet: È il contenuto informativo vero e proprio (es. "Posizione asse", "Velocità" o "Temperatura").

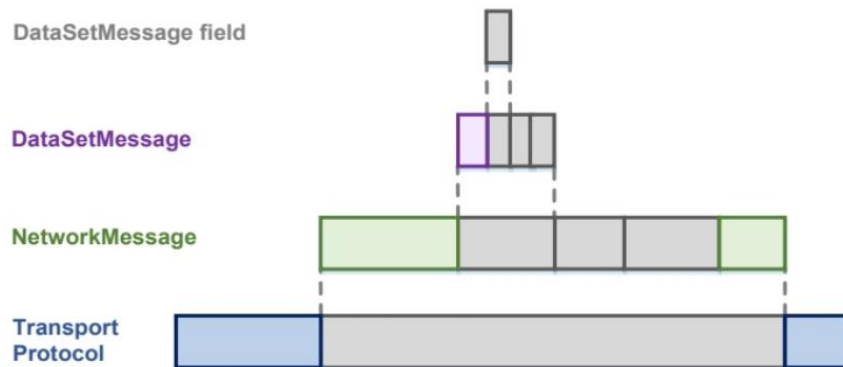


Figura 10 - OPC UA FX message layer

Il vantaggio della codifica UADP consiste nella sua compattezza. A differenza dei formati autodescrittivi come JSON o XML, dove ogni valore è accompagnato dal suo nome (es. {"temperatura": 24.5}), in UADP le meta-informazioni non viaggiano nel pacchetto dati. Publisher e Subscriber condividono una conoscenza a priori della struttura del messaggio, stabilita in fase di configurazione (codificata nel codice, tramite file descrittori o scambio iniziale). Il significato di un dato è quindi determinato esclusivamente dalla sua posizione (offset) all'interno del payload binario e non da un'etichetta testuale. Si immagini un DataSet composto da due variabili in virgola mobile (Float, 4 byte ciascuna): "Velocità" e "Posizione". In un pacchetto JSON, sarebbe necessario trasmettere circa 40-50 byte di caratteri ASCII per descrivere nomi e valori. In un pacchetto UADP, grazie all'accordo preventivo, il Subscriber sa che:

- I byte da 0 a 3 corrispondono sempre alla "Velocità".
- I byte da 4 a 7 corrispondono sempre alla "Posizione".

Il payload trasmesso sarà quindi di soli 8 byte di dati grezzi. Questa eliminazione dell'overhead minimizza l'uso della banda e riduce il tempo di parsing della CPU, rendendo la codifica ideale per il determinismo richiesto da TSN.^[11]

2.2 Virtualizzazione e Containerizzazione nel Cloud Continuum

All'interno dell'infrastruttura del Cloud Continuum, dove le applicazioni devono poter migrare dal Cloud centrale ai dispositivi Edge, la virtualizzazione basata su macchine virtuali (VM) è inefficiente a causa dell'elevata quantità di risorse necessarie. La tecnologia più adatta in questa situazione è infatti la containerizzazione.

2.2.1 Dalle virtual machine ai container

A differenza delle VM che richiedono un intero sistema operativo (Guest OS) per ogni istanza di virtualizzazione, i container condividono il kernel del sistema operativo che li ospita (Host OS). Solo i processi, il file system e le risorse di rete vengono isolati dal container. Questo approccio garantisce una maggior leggerezza: il container occupa solo pochi megabyte, a differenza dei gigabyte occupati dalle VM. Anche l'avvio ne beneficia richiedendo solo qualche millisecondo visto che non è necessario effettuare il boot di un sistema operativo. L'aspetto più importante è però la portabilità: i container permettono di scrivere il software una sola volta ed eseguirlo in modo identico su qualsiasi dispositivo (dal server Cloud al nodo Edge), grazie al fatto che l'applicazione è impacchettata insieme a tutte le sue dipendenze e librerie.^[12]

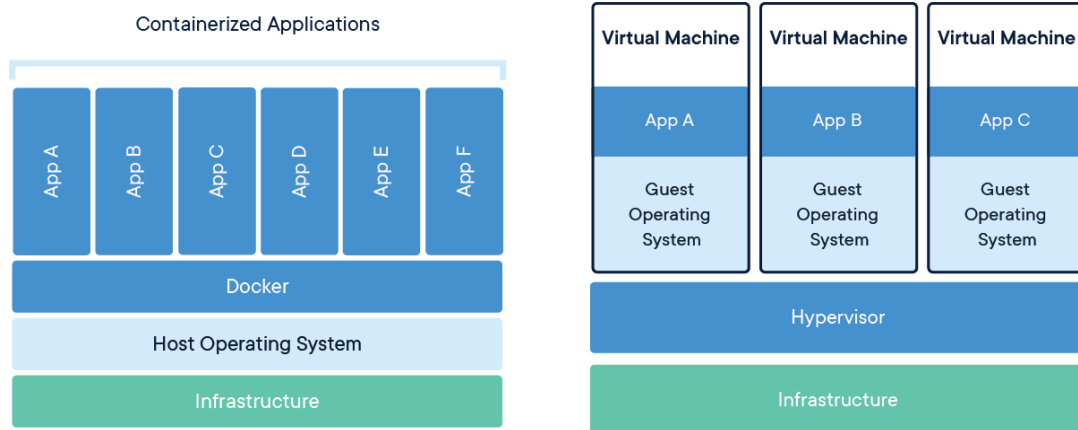


Figura 11 - Differenza tra container e VM⁵

2.2.2 Docker

La tecnologia più utilizzata per la gestione dei container è Docker. Docker introduce il concetto di immagine cioè un pacchetto immutabile che contiene tutto il necessario per l'esecuzione dell'applicazione.



Figura 12 – Logo Docker

Tuttavia, l'uso di Docker in ambito industriale e TSN richiede attenzione specifica alla configurazione di rete. Di default, Docker isola la rete del container tramite un bridge virtuale (NAT). Questa configurazione è problematica per il Real-Time poiché aggiunge un livello software che può aumentare la latenza e mascherare il traffico, rendendo complesso l'utilizzo di protocolli Layer 2 come IEEE 802.1Q. Per applicazioni che necessitano di inviare pacchetti Multicast o accedere alle proprietà della VLAN è spesso necessario configurare modalità di rete avanzate (come host network o driver macvlan) per bypassare l'isolamento e accedere direttamente all'interfaccia fisica.^[12]

⁵ Foto: <https://www.docker.com/resources/what-container/>

2.2.3 Orchestrazione e Kubernetes

Docker si occupa di gestire il ciclo di vita del singolo container, in scenari complessi (es. flotte di centinaia di Digital Twin) è però necessario un sistema di orchestrazione. Kubernetes (K8s) è lo standard per l'automazione del deployment e della gestione di applicazioni containerizzate. Kubernetes rappresenta l'infrastruttura che permette di spostare dinamicamente il Digital Twin dall'Edge al Cloud e viceversa in base al carico di lavoro o alla latenza di rete rilevata.

2.3 Time Sensitive Networking (TSN)

Il Time Sensitive Networking (TSN) è un insieme di standard sviluppati per estendere le capacità di Ethernet. L'obiettivo di questa tecnologia è quello di permettere la convergenza tra le reti IT e OT. Invece di mantenere reti fisiche separate per il controllo delle macchine e per la gestione dei dati, TSN permette di unificare tutto in una singola infrastruttura, garantendo che il traffico critico non venga rallentato da quello non critico.

Per raggiungere questo livello di determinismo, TSN si basa su due meccanismi principali: la sincronizzazione temporale e la gestione delle priorità.

2.3.1 Sincronizzazione Temporale (IEEE 802AS)

Il prerequisito fondamentale per operare in maniera deterministica è che tutti i dispositivi della rete (switch, sensori, controllori) condividano la stessa base temporale globale. Senza un riferimento temporale comune non è possibile coordinare azioni distribuite.

Lo standard di riferimento per sincronizzare i dispositivi è l'IEEE 802.1AS, noto anche come gPTP cioè generalized Precision Time Protocol. Il funzionamento si basa su una gerarchia Master/Slave e un meccanismo continuo di correzione dei ritardi. La rete elegge un Grandmaster Clock che sarà il dispositivo fonte globale del tempo. Questo

nodo invia periodicamente messaggi di sincronizzazione (Sync Message) a tutta la rete.

È necessaria però una correzione continua del tempo. Il punto critico in una rete industriale è il tempo che un pacchetto impiega per attraversare uno switch (tempo di elaborazione e accodamento). Gli switch compatibili con 802.1AS agiscono come Transparent Clocks: misurano esattamente quanto tempo il messaggio di Sync è rimasto al loro interno e scrivono questo valore in un campo speciale del pacchetto (o in un messaggio di Follow_Up successivo). Il protocollo 802.1AS richiede che ogni porta di ogni dispositivo misuri costantemente il ritardo di propagazione fisico del cavo che la collega al dispositivo vicino (Link Delay), scambiando messaggi Pdelay_Req e Pdelay_Resp. Grazie alla somma di tutti questi contributi ogni nodo può calcolare l'ora esatta.

Per evitare loop di sincronizzazione ogni porta deve assumere un ruolo specifico. La slave port è la porta che guarda verso il Grandmaster. Riceve i messaggi di Sync e li utilizza per allineare l'orologio locale del dispositivo. Ogni nodo può avere una sola Slave Port. La Master Port, invece, si occupa di propagare il tempo verso i dispositivi a valle. Una volta che il dispositivo si è sincronizzato, rigenera i messaggi di tempo e li invia tramite le sue porte master verso controller, robot o altri switch. ^[9]

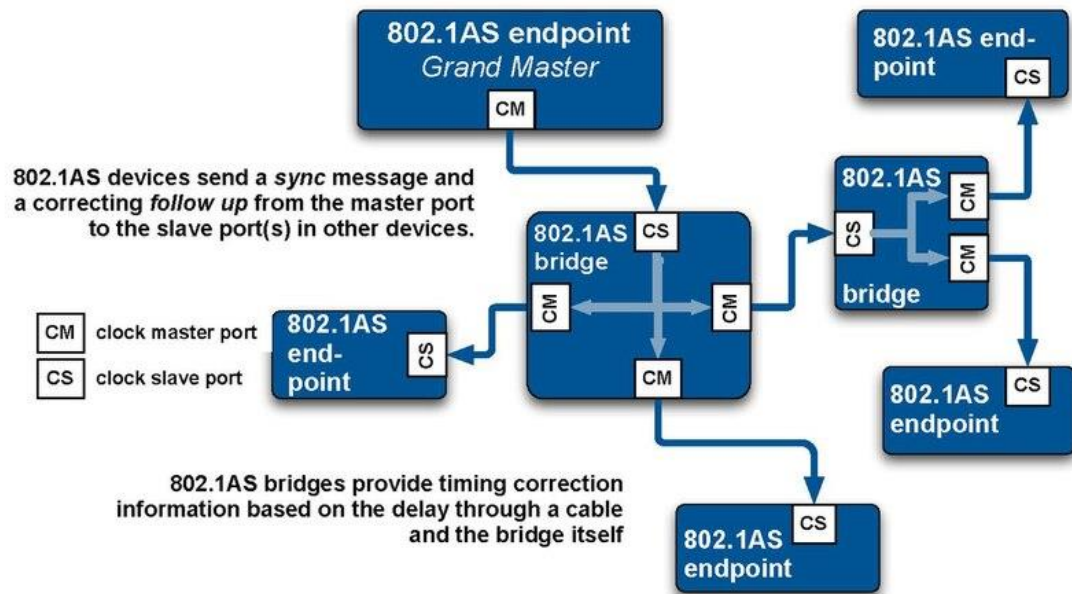


Figura 13 - Schema IEEE 802.1AS⁶

È fondamentale distinguere l'approccio deterministico di IEEE 802.1AS dai protocolli di sincronizzazione tradizionali come NTP (Network Time Protocol). L'NTP è una soluzione completamente software che genera i timestamp a livello applicativo. Di conseguenza, il pacchetto di sincronizzazione deve attraversare l'intero stack di rete (UDP/IP) e subire le latenze variabili introdotte dal sistema operativo. Questo limita quindi la precisione di NTP nell'ordine dei millisecondi. Al contrario, lo standard IEEE 1588 (su cui si basa 802.1AS) supera questo limite introducendo l'Hardware Timestamping. La marcatura temporale non avviene via software, ma viene impressa fisicamente dal chip di rete nell'istante esatto in cui il primo bit del messaggio lascia l'interfaccia. Inoltre, grazie al supporto hardware gli switch correggono i ritardi di propagazione lungo la rete, raggiungendo una precisione di sincronizzazione nell'ordine dei nanosecondi.^[14]

⁶ Foto: <https://en.wikipedia.org/wiki/File:Clock-Master.pdf>

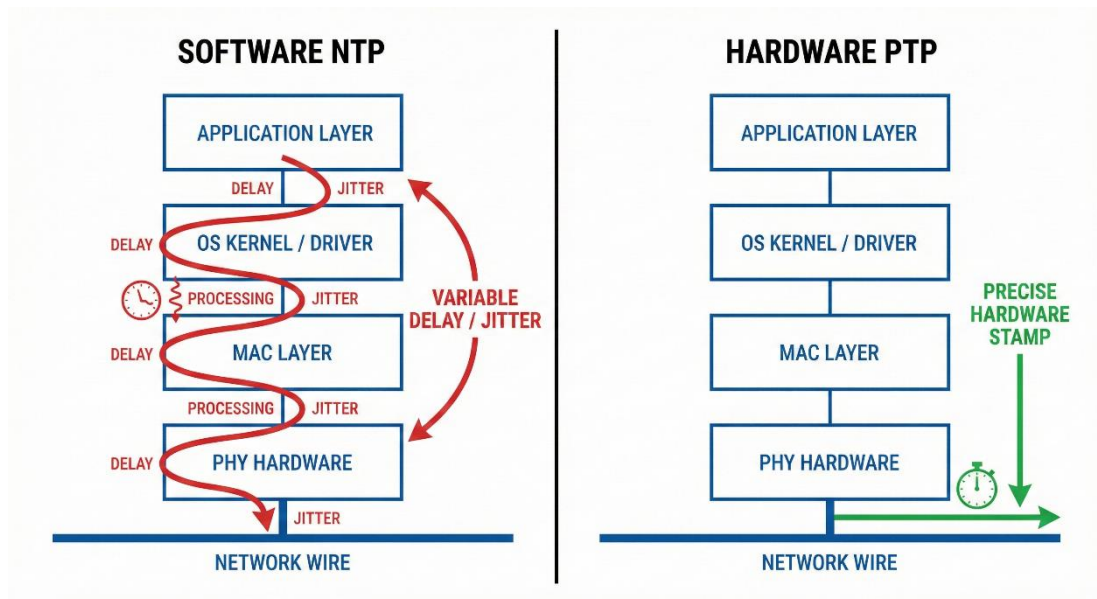


Figura 14 - NTP VS PTP

2.3.2 Prioritizzazione e scheduling (IEEE 802.1Qbv)

Il secondo pilastro del TSN riguarda la gestione deterministica del traffico. In una rete Ethernet convenzionale, la gestione delle code avviene tramite priorità definite dallo standard IEEE 802.1Q. Questo standard utilizza il campo PCP (Priority Code Point) inserito nell'header del tag VLAN.

Sebbene la prioritizzazione stretta (Strict Priority) offerta dalle VLAN sia spesso sufficiente per applicazioni Soft Real-Time essa presenta un limite fisico: se un pacchetto a bassa priorità inizia la trasmissione appena prima che arrivi un pacchetto critico, quest'ultimo deve attendere la fine della trasmissione in corso.

Per risolvere questo problema nei casi Hard Real-Time, TSN introduce il Time Aware Shaper (TAS). Il protocollo trasforma lo switch in un dispositivo a divisione di tempo (TDMA) che si basa su tre concetti chiave:

- **Transmission Gate:** Ogni coda di uscita dello switch è dotata di un cancello. Quando il gate è open i pacchetti della coda possono essere trasmessi. Quando il gate è closed rimangono nel buffer.

- Gate Control List: definisce ciclicamente quale gate deve essere aperto e per quanto tempo. GCL è sincronizzato con il tempo della rete (gPTP) in modo che tutti gli switch aprano la corsia preferenziale nello stesso momento.
- Guard Bands: Si tratta di un meccanismo di sicurezza fondamentale. Per evitare che un pacchetto Best Effort invada la finestra temporale riservata al traffico critico, lo switch calcola se la trasmissione può essere completata prima della chiusura del Gate. Se il tempo residuo non è sufficiente la trasmissione viene bloccata in anticipo. La banda di guardia garantisce che quando la finestra del Digital twin si apre il mezzo di trasmissione sia libero.

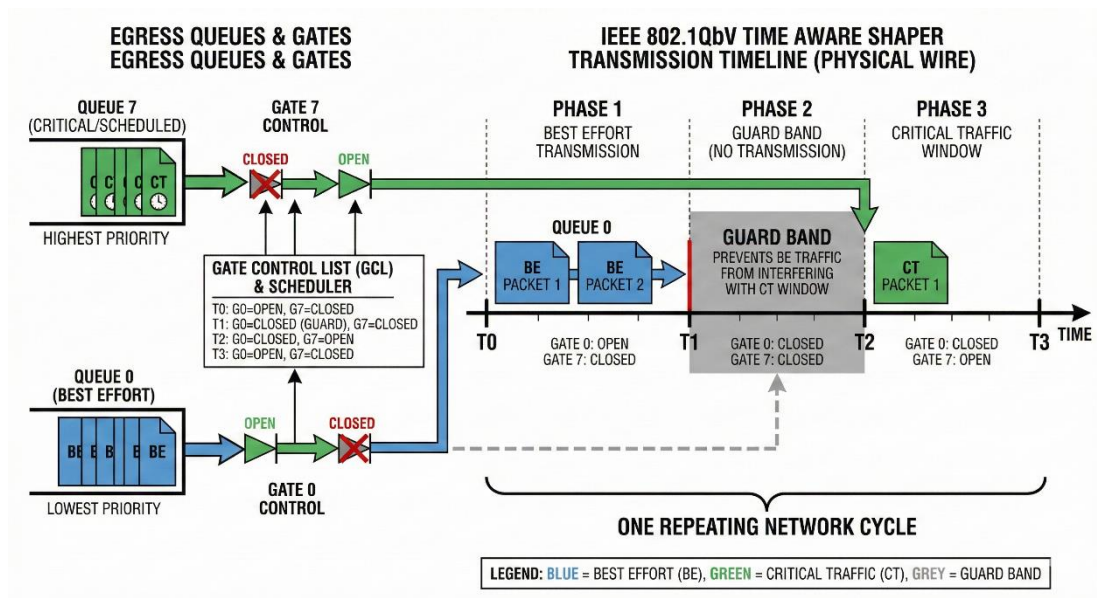


Figura 15 - Schema IEEE 802.1Qbv

Capitolo 3: Architettura del Sistema ASTRID

3.1 Architettura generale del sistema

L'architettura proposta, denominata ASTRID (Advanced Support for Time-sensitive Reliable Industrial Digital-twins) mira a replicare uno scenario industriale reale, implementando il paradigma OPC UA Field eXchange (FX) su una rete Time Sensitive Networking (TSN). Il sistema è stato progettato per valutare la convergenza tra il traffico IT (monitoraggio e configurazione) e il traffico OT (controllo Real-Time) su un'unica infrastruttura fisica e garantendo una sincronizzazione al nanosecondo.

3.1.1 Topologia fisica

L'infrastruttura hardware alla base di ASTRID è basata su una topologia a stella con lo switch TSN al centro. Questa configurazione riduce al minimo gli hop. Ci sono tre nodi coinvolti. Il Publisher: un PC industriale Advantech (x86) che ha il ruolo di emulare il controllore di un braccio robotico generando dati di telemetria. Inoltre, è configurato come IEEE 802.1AS Grandmaster. Lo switch: un Relyum TSN Bridge che si occupa di instradare i pacchetti, correggere i ritardi di propagazione e gestire la VLAN. Nella rete PTP opera come Transparent Bridge. Il modulo NVIDIA Jetson AGX Orin (Arm) svolge il ruolo di Digital Twin (Subscriber). Riceve i dati ad alta

frequenza e li archivia su InfluxDB. Nella rete PTP è uno slave, sincronizza quindi il proprio orologio con quello dell'Advantech.

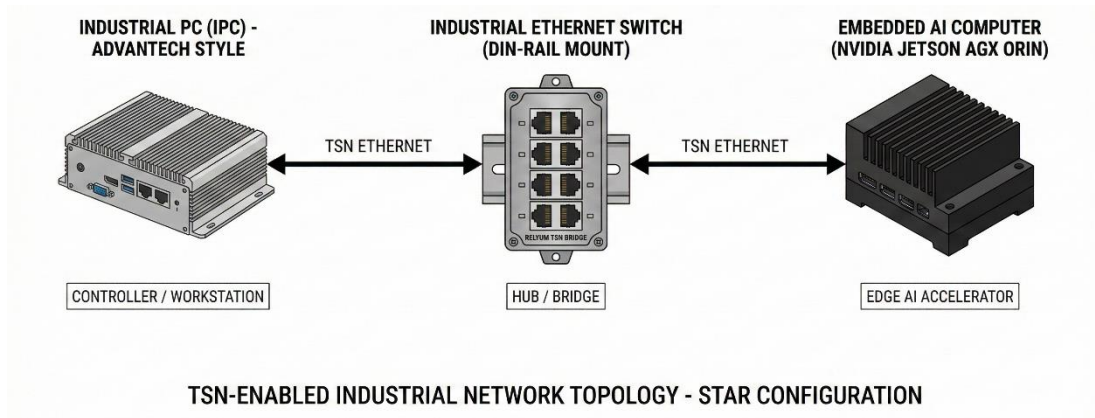


Figura 16 - Topologia fisica

3.1.2 Topologia Logica

Dal punto di vista software, ASTRID implementa un'architettura ibrida conforme alle specifiche OPC UA FX. Sono presenti due canali di comunicazione: uno Real-Time e uno no. Il canale che opera in tempo reale utilizza OPC UA PubSub con codifica binaria UADP su trasporto UDP. I dati vengono pubblicati ogni 1ms e contengono: posizione degli assi, vibrazione e temperatura. Il traffico viene mappato con priorità PCP 6 e isolato sulla VLAN 100. Il flusso è unidirezionale tra Publisher e Subscriber. Il secondo canale di comunicazione usa OPC UA Client/Server su trasporto TCP. Permette un accesso on-demand alle informazioni del dispositivo. Rende possibile usare client esterni come UaExpert per modificare parametri di funzionamento come la velocità del macchinario emulato. Il flusso è bidirezionale.

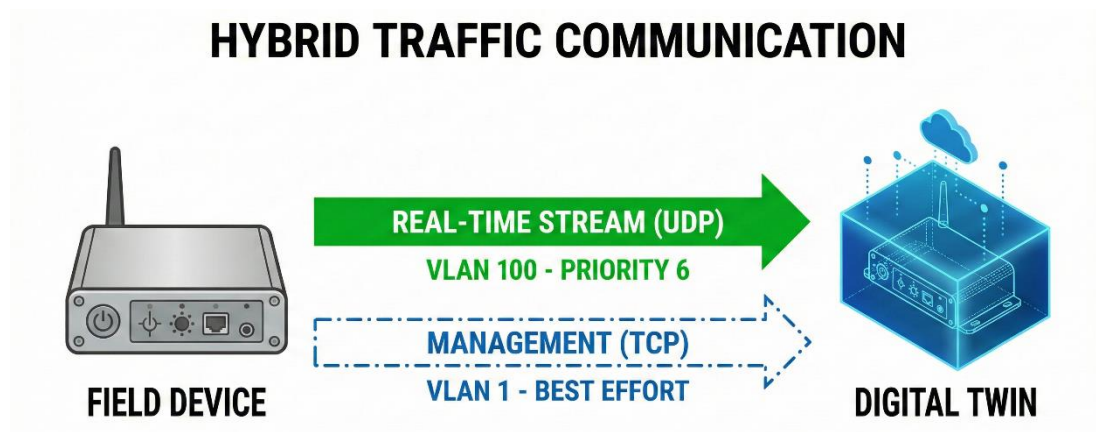


Figura 17 – Topologia logica

3.1.3 Strategie di deployment

Per eseguire i software è stato scelto il deployment Bare Metal scartando l'ipotesi della containerizzazione. Questa scelta è stata fatta per la necessità di accedere direttamente al livello più basso del kernel Linux e dell'hardware di rete. L'accesso ai driver della scheda di rete è indispensabile per permettere ai demoni PTP (ptp41) di marcare i pacchetti a livello fisico evitando l'imprecisione dello stack software. Inoltre, l'applicazione dei tag di priorità IEEE 802.1Q tramite iptables richiede un controllo sull'interfaccia di rete fisica che verrebbe oscurato dai layer di astrazione di Docker.

3.2 Configurazione dell'infrastruttura di rete

Una volta definita la struttura logica e fisica dell'infrastruttura, è stato necessario configurare lo strato di rete in grado di supportare il determinismo. A differenza delle tradizionali reti Ethernet, una rete TSN richiede una serie di passaggi per permettere al traffico di fluire correttamente.

3.2.1 Configurazione delle interfacce virtuale (VLAN)

Per isolare il traffico Real-Time da quello Best-Effort sono state create delle interfacce virtuali sui nodi terminali (Advantech e Orin). Con l'uso del comando:

ip link add link <iface> name <iface>.100 type vlan id 100

è stato possibile creare interfacce virtuali etichettate con VLAN id 100. È stata poi definita una subnet dedicata esclusivamente al traffico PubSub (192.168.99.x)

```
9: enp4s0.100@enp4s0: <BROADCAST,MULTICAST,UP,LOWER UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether cc:82:7f:33:32:18 brd ff:ff:ff:ff:ff:ff
    inet 192.168.99.1/24 scope global enp4s0.100
        valid_lft forever preferred_lft forever
    inet6 fe80::ce82:7fff:fe33:3218/64 scope link
        valid_lft forever preferred_lft forever
11: eth1.100@eth1: <BROADCAST,MULTICAST,UP,LOWER UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 00:30:64:74:aa:92 brd ff:ff:ff:ff:ff:ff
    inet 192.168.99.2/24 scope global eth1.100
        valid_lft forever preferred_lft forever
    inet6 fe80::230:64ff:fe74:aa92/64 scope link
        valid_lft forever preferred_lft forever
```

Figura 18 - Vlan: in alto Advantech, in basso Orin

3.2.2 Configurazione dello switch

Tramite la modalità Trunk lo switch è stato configurato per gestire il passaggio di due flussi di dati paralleli. È stata impostata una VLAN 1 (Management) che gestisce il traffico SSH configurata come Untagged per garantire la connettività con i PC. La rete VLAN 100 dedicata al traffico critico è stata configurata come Tagged Member. Questa impostazione costringe lo switch a mantenere l'header 802.1Q durante l'inoltro del pacchetto verso il Subscriber.

VLAN CONFIGURATION Port Configuration (SWITCH)							
VLAN enabled ☒							
	PVID	PCP	Port type	Acceptable frame types	Ingress filtering	Egress tagging	Hybrid port type
PORT_0	1	0	Trunk	Admit all	True	Untag port	
PORT_1	1	0	Trunk	Admit all	True	Untag port	
PORT_2	1	0	Trunk	Admit all	True	Untag port	
PORT_3	1	0	Trunk	Admit all	True	Untag port	
INTERNAL_PORT	1	0	Trunk	Admit all	True	Untag port	

VLAN CONFIGURATION VLAN table					
	VID	VLAN name	Egress ports members	Untagged ports members	Edit
☐	3	test	SWITCH/1		☒
☐	100	TSN_Robot	SWITCH/0-1		☒

Figura 19 - Vlan configuration

Oltre all'isolamento tramite la VLAN, per garantire maggior determinismo è stato configurato anche il Time Aware Shaper (TAS). Lo switch opera con un ciclo di

1.000.000 ns cioè 1 ms. All'interno di questo ciclo è stata definita una Gate Control List (GCL) per gestire le code di trasmissione. Sono state create due finestre temporali. Una della durata di 400.000 ns cioè 400 microsecondi per la coda ad alta priorità (Q6) che è l'unica aperta mentre le altre code sono chiuse. Una seconda della durata di 600.000 ns cioè 600 microsecondi in cui il gate della coda Q6 è chiuso mentre le altre code del traffico non critico sono aperte.

The screenshot shows the 'TIME AWARE SHAPER | Administrative' interface. It contains several configuration fields and a table for the Gate Control List (GCL).

Configuration Fields:

- Gate states: All queues (Q7 to Q0) are checked.
- Cycle time (ns): 1000000
- Cycle time extension (ns): 10
- Base time seconds: 1
- Base time nanoseconds: 1
- Control list length: 2

GCL Table:

Time Interval (ns)	Slot/Queue	Q7	Q6	Q5	Q4	Q3	Q2	Q1	Q0
400000	Slot 0	☐	☒	☐	☐	☐	☐	☐	☐
600000	Slot 1	☒	☐	☒	☒	☒	☒	☒	☒

Figura 20 - Configurazione TAS

3.2.3 Prioritizzazione del traffico

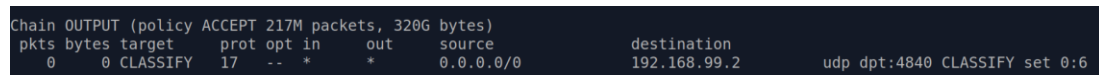
Le applicazioni Publisher e Subscriber sono state scritte in Python. Questo linguaggio lavora in user space ed è quindi legato ad un concetto di tempo relativo. Utilizzando primitive asincrone come `asyncio.sleep(0.001)` per pubblicare dati ogni millisecondo il programma opera in un regime di Soft-RealTime visto che dipende dallo scheduling del sistema operativo che impiegherà almeno un millisecondo per completare l'operazione. L'invio del pacchetto non avviene quindi mai esattamente ogni 1 millisecondo ma è soggetto ad un jitter software imprevedibile.

Questa limitazione non ha quindi permesso di adottare meccanismi avanzati di controllo del traffico messi a disposizione da Linux. Non è stato possibile lavorare con le queuing disciplines taprio o etf (Earliest TxTime First). L'algoritmo etf, infatti, non

accetta comandi di invio immediato ma richiede che l'applicazione fornisca un timestamp di trasmissione sincronizzato con la rete PTP. Sulla base di questo tempo, la scheda di rete apre un gate di trasmissione per una finestra temporale molto breve. Il ritardo accumulato dalla sleep, però, non avrebbe permesso ai pacchetti generati da Python di raggiungere la scheda nel momento in cui i gate venivano aperti causando l'accodamento o lo scarto dei pacchetti.

Non operando così a basso livello e non potendo tramite Python assegnare a un pacchetto il tag di priorità la soluzione adottata è stata quella di utilizzare le iptables. È stata creata una regola nella tabella mangle per intercettare i pacchetti UDP diretti alla porta 4840 e marcarli con la classe di servizio 6:

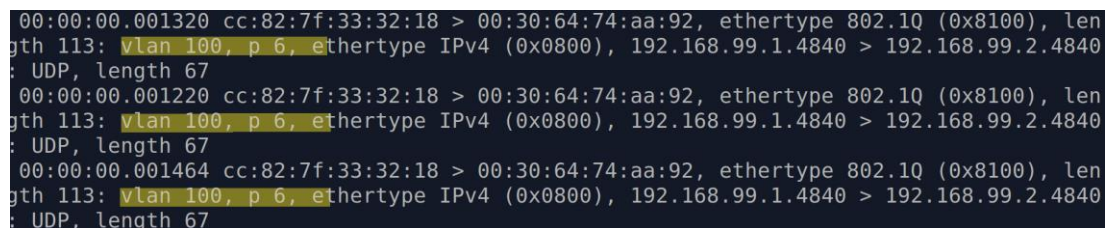
```
sudo iptables -t mangle -A OUTPUT -p udp -d 192.168.99.2 --dport 4840 -j CLASSIFY --set-class 0:6
```



Chain OUTPUT (policy ACCEPT 217M packets, 320G bytes)									
pkts	bytes	target	prot	opt	in	out	source	destination	
0	0	CLASSIFY	17	--	*	*	0.0.0.0/0	192.168.99.2	udp dpt:4840 CLASSIFY set 0:6

Figura 21 - Tabella mangle

Questa regola mappa il traffico generato dallo script del Publisher sulla priorità PCP 6. I test effettuati con tcpdump hanno confermato che i pacchetti in uscita presentano il campo p 6 nell'header 802.1Q.



```
00:00:00.001320 cc:82:7f:33:32:18 > 00:30:64:74:aa:92, ethertype 802.1Q (0x8100), length 113: vlan 100, p 6, ethertype IPv4 (0x0800), 192.168.99.1.4840 > 192.168.99.2.4840: UDP, length 67
00:00:00.001220 cc:82:7f:33:32:18 > 00:30:64:74:aa:92, ethertype 802.1Q (0x8100), length 113: vlan 100, p 6, ethertype IPv4 (0x0800), 192.168.99.1.4840 > 192.168.99.2.4840: UDP, length 67
00:00:00.001464 cc:82:7f:33:32:18 > 00:30:64:74:aa:92, ethertype 802.1Q (0x8100), length 113: vlan 100, p 6, ethertype IPv4 (0x0800), 192.168.99.1.4840 > 192.168.99.2.4840: UDP, length 67
```

Figura 22 - Tcpdump priorità vlan

3.3 Sincronizzazione temporale (IEEE 802.1AS)

Per operare con determinismo è necessario che le macchine condividano una base temporale comune. Si è scelto di utilizzare lo standard IEEE 802.1AS (gPTP), ottimizzato per reti locali a alta precisione, preferendolo al classico NTP che operando via software offre una precisione solo nell'ordine dei millisecondi.

3.3.1 Gerarchia PTP

La topologia di sincronizzazione è stata definita configurando il parametro `priority1` nel file di configurazione di PTP (`gptp.cfg`). L'Advantech è il Grandmaster con `priority1 = 5`, lo switch ha invece `priority1 = 10` ed infine l'Orin essendo lo slave ha il valore più alto `priority1 = 250` per inibire l'elezione del master.

```
[global]
gmCapable           1
priority1           10
priority2           248
logAnnounceInterval 0
logSyncInterval     -3
syncReceiptTimeout  3
neighborPropDelayThresh 800
min_neighbor_prop_delay -20000000
path_trace_enabled  1
follow_up_info       1
transportSpecific    1
ptp_dst_mac         01:80:C2:00:00:0E
network_transport    L2
delay_mechanism      P2P
```

Figura 23 - `gptp.cfg`

3.3.2 Implementazione con LinuxPTP

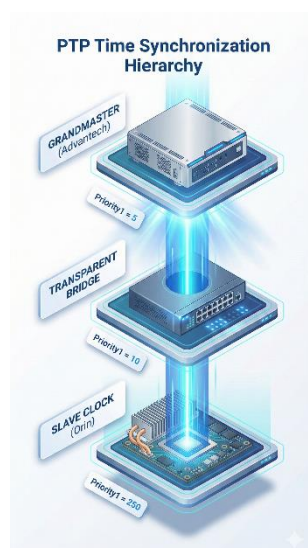
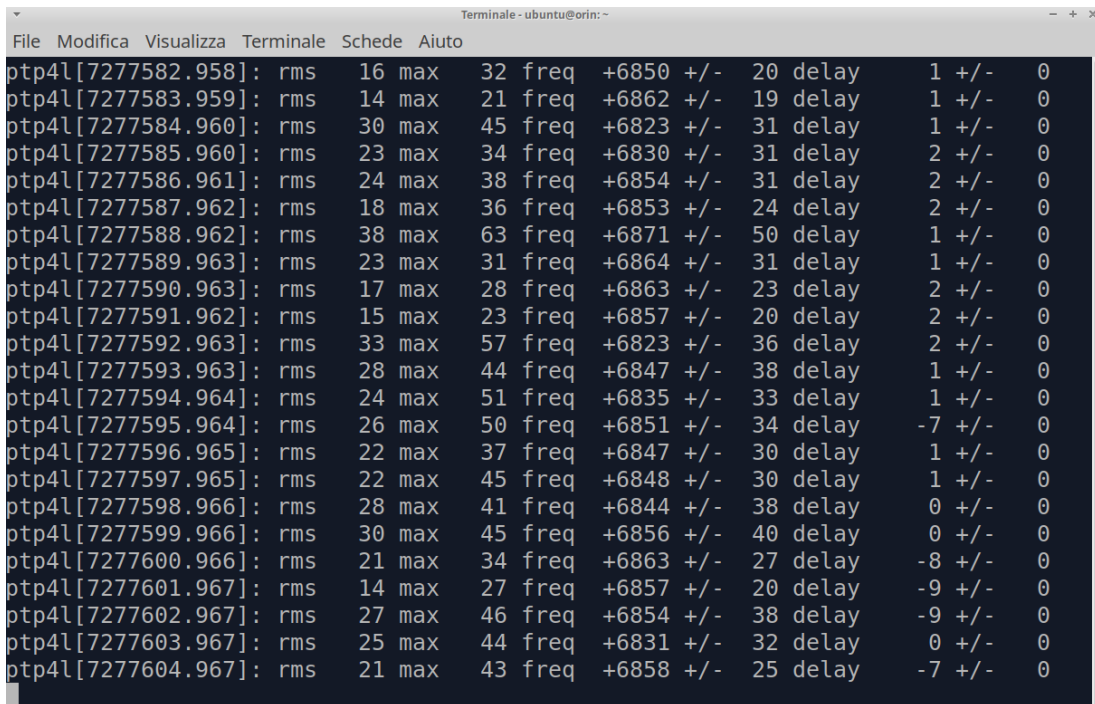


Figura 24 - PTP Hierarchy

Sui nodi Linux è stato usato il set di strumenti `linuxptp` per eseguire due demoni che lavorano parallelamente. `Ptp4l` rappresenta il protocol layer e gestisce lo scambio di pacchetti Sync e Follow_Up sul cavo Ethernet. Inoltre, sincronizza l'orologio della scheda di rete.

```
sudo ptp4l -i <interface> -f gptp.cfg -m
```



```
ptp4l[7277582.958]: rms 16 max 32 freq +6850 +/- 20 delay 1 +/- 0
ptp4l[7277583.959]: rms 14 max 21 freq +6862 +/- 19 delay 1 +/- 0
ptp4l[7277584.960]: rms 30 max 45 freq +6823 +/- 31 delay 1 +/- 0
ptp4l[7277585.960]: rms 23 max 34 freq +6830 +/- 31 delay 2 +/- 0
ptp4l[7277586.961]: rms 24 max 38 freq +6854 +/- 31 delay 2 +/- 0
ptp4l[7277587.962]: rms 18 max 36 freq +6853 +/- 24 delay 2 +/- 0
ptp4l[7277588.962]: rms 38 max 63 freq +6871 +/- 50 delay 1 +/- 0
ptp4l[7277589.963]: rms 23 max 31 freq +6864 +/- 31 delay 1 +/- 0
ptp4l[7277590.963]: rms 17 max 28 freq +6863 +/- 23 delay 2 +/- 0
ptp4l[7277591.962]: rms 15 max 23 freq +6857 +/- 20 delay 2 +/- 0
ptp4l[7277592.963]: rms 33 max 57 freq +6823 +/- 36 delay 2 +/- 0
ptp4l[7277593.963]: rms 28 max 44 freq +6847 +/- 38 delay 1 +/- 0
ptp4l[7277594.964]: rms 24 max 51 freq +6835 +/- 33 delay 1 +/- 0
ptp4l[7277595.964]: rms 26 max 50 freq +6851 +/- 34 delay -7 +/- 0
ptp4l[7277596.965]: rms 22 max 37 freq +6847 +/- 30 delay 1 +/- 0
ptp4l[7277597.965]: rms 22 max 45 freq +6848 +/- 30 delay 1 +/- 0
ptp4l[7277598.966]: rms 28 max 41 freq +6844 +/- 38 delay 0 +/- 0
ptp4l[7277599.966]: rms 30 max 45 freq +6856 +/- 40 delay 0 +/- 0
ptp4l[7277600.966]: rms 21 max 34 freq +6863 +/- 27 delay -8 +/- 0
ptp4l[7277601.967]: rms 14 max 27 freq +6857 +/- 20 delay -9 +/- 0
ptp4l[7277602.967]: rms 27 max 46 freq +6854 +/- 38 delay -9 +/- 0
ptp4l[7277603.967]: rms 25 max 44 freq +6831 +/- 32 delay 0 +/- 0
ptp4l[7277604.967]: rms 21 max 43 freq +6858 +/- 25 delay -7 +/- 0
```

Figura 25 - ptp4l

Analizzando il log generato dal comando ptp4l possiamo osservare che la sincronizzazione dei sistemi sta avvenendo correttamente. Il Root Mean Square (rms) rappresenta la media degli scarti quadratici medi tra l'orologio locale e quello del Grandmaster. I valori oscillano tra 15 e 30 nanosecondi. L'errore massimo per ogni intervallo di campionamento si mantiene sotto i 65 nanosecondi. La frequenza mostra invece la correzione applicata per sopperire al drift naturale del quarzo. Infine, delay rappresenta il ritardo di propagazione sul cavo.

Phc2sys è il layer di sistema. Si occupa di allineare l'orologio del sistema con l'orologio hardware della scheda di rete.

- Sul master: Copia dal sistema alla scheda: `sudo phc2sys -c enp4s0 -s CLOCK_REALTIME -O 0 -m`
- Sullo slave: Copia dalla scheda al sistema: `sudo phc2sys -s eth1 -c CLOCK_REALTIME -O 0 -m`

Dai log di sistema è stato possibile verificare il funzionamento dell'algoritmo con messaggi quali "assuming the grand master role" sul Publisher e "selected best master clock" sul Subscriber.

```
Terminale - ubuntu@edge2-arm-f: ~
File Modifica Visualizza Terminale Schede Aiuto
phc2sys[7665793.003]: enp4s0 sys offset -3 s2 freq -11869 delay 0
phc2sys[7665794.004]: enp4s0 sys offset 2 s2 freq -11865 delay 0
phc2sys[7665795.004]: enp4s0 sys offset 7 s2 freq -11859 delay 0
phc2sys[7665796.004]: enp4s0 sys offset 0 s2 freq -11864 delay 0
phc2sys[7665797.004]: enp4s0 sys offset 5 s2 freq -11859 delay 0
phc2sys[7665798.004]: enp4s0 sys offset 2 s2 freq -11861 delay 0
phc2sys[7665799.005]: enp4s0 sys offset 1 s2 freq -11861 delay 0
phc2sys[7665800.005]: enp4s0 sys offset 1 s2 freq -11861 delay 0
phc2sys[7665801.005]: enp4s0 sys offset -3 s2 freq -11864 delay 0
phc2sys[7665802.005]: enp4s0 sys offset 4 s2 freq -11858 delay 0
phc2sys[7665803.005]: enp4s0 sys offset 2 s2 freq -11859 delay 0
phc2sys[7665804.006]: enp4s0 sys offset -5 s2 freq -11865 delay 0
phc2sys[7665805.006]: enp4s0 sys offset -5 s2 freq -11867 delay 0
phc2sys[7665806.006]: enp4s0 sys offset -1 s2 freq -11864 delay 0
phc2sys[7665807.006]: enp4s0 sys offset -4 s2 freq -11868 delay 0
phc2sys[7665808.006]: enp4s0 sys offset -12 s2 freq -11877 delay 0
phc2sys[7665809.007]: enp4s0 sys offset 0 s2 freq -11869 delay 0
phc2sys[7665810.007]: enp4s0 sys offset 3 s2 freq -11866 delay 0
phc2sys[7665811.007]: enp4s0 sys offset 3 s2 freq -11865 delay 0
phc2sys[7665812.007]: enp4s0 sys offset 4 s2 freq -11863 delay 0
phc2sys[7665813.008]: enp4s0 sys offset 0 s2 freq -11866 delay 0
phc2sys[7665814.008]: enp4s0 sys offset -2 s2 freq -11868 delay 0
phc2sys[7665815.008]: enp4s0 sys offset 1 s2 freq -11865 delay 0
```

Figura 26 - phc2sys

Il log di phc2sys mostra la differenza temporale tra l'orologio della scheda di rete enp4s0 e il kernel Linux. Il delay è a zero perché la comunicazione avviene all'interno dei bus hardware del sistema.

In sintesi, l'errore di ptp4l minore di 65 ns e l'errore di phc2sys minore di 10 ns garantiscono un errore massimo inferiore al microsecondo.

3.4 Implementazione software

Il software implementa le specifiche descritte dalla documentazione di OPC UA FX. Il core applicativo di ASTRID realizza un sistema ibrido che unisce la flessibilità della configurazione client/server con la velocità del pattern PubSub.

3.4.1 Publisher: architettura ibrida

Il nodo Publisher è stato sviluppato con un'architettura ibrida che implementa il modello base di Automation Component descritto nella part 14 della documentazione di OPC UA. Per lo sviluppo del software Publisher e Subscriber è stato scelto Python.

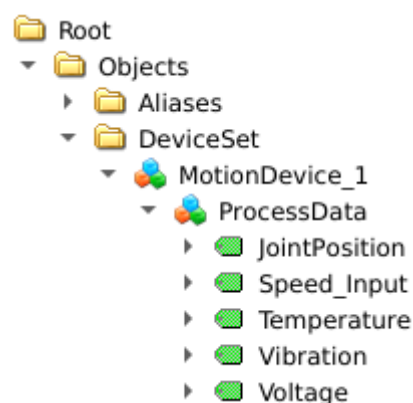


Figura 27 - Struttura dati visibile su UaExpert

Questa decisione è stata presa visto l'ampio numero di librerie disponibili, l'ampia agilità che offre nella fase di prototipazione e la forte diffusione del framework opcua-asyncio. L'utilizzo della primitiva asincrona `asyncio.sleep` permette di gestire la concorrenza tra i task anche se opera in regime Soft-RealTime. Analizzando il codice sorgente, il componente gestisce due canali di comunicazione paralleli all'interno dello stesso Event Loop asincrono: canale di controllo OPC

UA su TCP e canale di streaming OPC UA FX su UDP.

Il primo espone un endpoint TCP (porta 4840) e un Address Space accessibile da client esterni. Questo permette di modificare a runtime i parametri di processo, in questo caso la variabile `Speed_Input`. Questo cambiamento influenza la logica di calcolo dei dati senza interrompere la trasmissione.

#	Server	Node Id	Display Name	Value	Datatype	Source Timestamp	Server Timestamp	Statuscode
1	@Advantech	NS2[Numeric]4	JointPosition	-43.1750113...	Double	17:01:24.798	17:01:24.798	Good
2	@Advantech	NS2[Numeric]5	Temperature	31.86069692...	Double	17:01:24.799	17:01:24.799	Good
3	@Advantech	NS2[Numeric]6	Vibration	0.099121572...	Double	17:01:24.799	17:01:24.799	Good
4	@Advantech	NS2[Numeric]7	Voltage	24.01079344...	Double	17:01:24.799	17:01:24.799	Good

Figura 28 - UaExpert

```

# =====
# 1. SERVER
# =====
# Il server permette di monitorare lo stato e ricevere comandi esterni.
server = Server()
await server.init()
server.set_endpoint("opc.tcp://0.0.0.0:4840")

# Creazione dello spazio dei nomi e della struttura oggetti
idx = await server.register_namespace("http://opcfoundation.org/UA/FX/AC/")
objects = server.nodes.objects

device_folder = await objects.add_folder(idx, "DeviceSet")
my_device = await device_folder.add_object(idx, "MotionDevice_1")
proc_data_folder = await my_device.add_object(idx, "ProcessData")

# --- DEFINIZIONE VARIABILI DATI
v_pos = await proc_data_folder.add_variable(idx, "JointPosition", 0.0, ua.VariantType.Double)
v_temp = await proc_data_folder.add_variable(idx, "Temperature", 25.0, ua.VariantType.Double)
v_vib = await proc_data_folder.add_variable(idx, "Vibration", 0.0, ua.VariantType.Double)
v_volt = await proc_data_folder.add_variable(idx, "Voltage", 24.0, ua.VariantType.Double)

# --- VARIABILE DI CONTROLLO (SCRIVIBILE da UaExpert) ---
v_speed = await proc_data_folder.add_variable(idx, "Speed_Input", 1.0, ua.VariantType.Double)
await v_speed.set_writable() # Permette a UaExpert di scrivere

```

Figura 29 - Codice server TCP

Il secondo permette la pubblicazione costante dei dati calcolati dall'algoritmo di simulazione fisica (calcolo di posizione sinusoidale, temperatura a dente di sega e vibrazione casuale).

```

# =====
# 3. IL LOOP
# =====
async with server:
    async with ps:
        start_time = time.time()

        # Variabili per la fisica
        current_phase = 0.0
        last_time = start_time

        while True:
            now_t = time.time()
            dt = now_t - last_time
            last_time = now_t
            now_ts = datetime.now(timezone.utc)

            # --- A. LEGGIAMO IL COMANDO DA UAEXPERT ---
            # Leggiamo la velocità desiderata impostata dall'utente via TCP
            speed_val = await v_speed.read_value()

            # Protezione: evita velocità nulle o negative
            if speed_val < 0.1: speed_val = 0.1

            # --- B. CALCOLO FISICA ---
            # Aggiorniamo la fase basandoci sulla velocità attuale
            # nuova_fase = vecchia_fase + (velocità * delta_tempo)
            current_phase += speed_val * dt

            # Calcoliamo il seno usando la fase accumulata
            val_pos = 100.0 * math.sin(current_phase)

            # Temperatura (Dente di sega)
            val_temp = 25.0 + (now_t % 60) * 0.5

            # Vibrazione (Rumore casuale amplificato dalla velocità)
            val_vib = random.uniform(0, 0.5) * speed_val

            # Voltaggio (Costante)
            val_volt = 24.0

            # --- C. INVIO DATI (PubSub UDP) ---
            ds_data = source.datasources[DS_NAME]
            ds_data["JointPosition"] = ua.DataValue(ua.Variant(val_pos, ua.VariantType.Double), SourceTimestamp=now_ts)
            ds_data["Temperature"] = ua.DataValue(ua.Variant(val_temp, ua.VariantType.Double), SourceTimestamp=now_ts)
            ds_data["Vibration"] = ua.DataValue(ua.Variant(val_vib, ua.VariantType.Double), SourceTimestamp=now_ts)
            ds_data["Voltage"] = ua.DataValue(ua.Variant(val_volt, ua.VariantType.Double), SourceTimestamp=now_ts)

            # --- D. AGGIORNAMENTO DATI TCP ---
            # Aggiorniamo i nodi del server per permettere la visualizzazione su UaExpert
            await v_pos.write_value(val_pos)
            await v_temp.write_value(val_temp)
            await v_vib.write_value(val_vib)
            # Non aggiorniamo v_speed

```

Figura 30 – Loop e calcolo fisica simulata

I dati generati vengono incapsulati in pacchetti UADP e trasmessi a frequenza controllata (ciclo di 1ms).

```

# =====
# 2. PUBSUB (UDP)
# =====

# Definizione della struttura dei dati (MetaData) che verrà spedita
ds_meta = pubsub.DataSetMeta.Create(DS_NAME)
ds_meta.add_scalar("JointPosition", ua.VariantType.Double)
ds_meta.add_scalar("Temperature", ua.VariantType.Double)
ds_meta.add_scalar("Vibration", ua.VariantType.Double)
ds_meta.add_scalar("Voltage", ua.VariantType.Double)

```

Figura 31 – Definizione metadati UADP

3.4.2 Subscriber: Decodifica e persistenza

Il nodo Subscriber implementa la logica di ricezione descritta nella Part 14 dello standard OPC UA. Utilizza `WriterGroupId`, `DataSetWriter` e `PublisherId` per selezionare il traffico pertinente. I metadati sono invece definiti staticamente nel codice simulando una connessione pre-configurata tipica dei dispositivi embedded privi di meccanismi di discovery complessi.

```

# Configurazione del ReaderGroup
# Controllo che i dati siano del publisher 1, writerGroup 1 e dataset 100
reader_group = pubsub.ReaderGroup.new(
  name="ReaderGroup1",
  enable=True,
  reader=[
    pubsub.DataSetReader.new(
      CFG.PublisherID,
      CFG.WriterGroupId,
      CFG.DataSetWriterId, # Leggo set di dati numero 100
      metadata,
      name="FXReader",
      subscribed=handler_ref,
      enabled=True,
    )
  ],
)

# Configurazione della connessione UDP
connection = pubsub.PubSubConnection.udp_uadp(
  "Subscriber Connection",
  ua.UInt16(1), # Ascolto dati del dispositivo con id 1
  UdpSettings(Url=CFG.Url),
  reader_groups=[reader_group],
)

```

Figura 32 - Filtro dati ingresso controllando WriterGroupId, DataSetWriter e PublisherId

```

# Definisce la struttura attesa del DataSet in ingresso
def create_meta_data():
  dataset = pubsub.DataSetMeta.Create("100")
  dataset.add_scalar("JointPosition", ua.VariantType.Double)
  dataset.add_scalar("Temperature", ua.VariantType.Double)
  dataset.add_scalar("Vibration", ua.VariantType.Double)
  dataset.add_scalar("Voltage", ua.VariantType.Double)
  return dataset

```

Figura 33 - Metadati UADP

Per garantire la coerenza temporale, il Subscriber non utilizza l'orario di ricezione del pacchetto, ma estrae il SourceTimestamp direttamente dal payload del messaggio UADP. Questo valore rappresenta l'istante esatto di generazione del dato lato Publisher, permettendo di ricostruire la storia reale dell'evento indipendentemente dalle latenze di rete.

```

# Callback invocata dalla libreria asyncua ogni volta che arriva un pacchetto UDP valido
async def on_dataset_received(self, _meta: pubsub.DataSetMeta, fields: list[pubsub.DataSetValue]):
    # Helper per estrarre il valore numerico puro
    def get_val(obj):
        while hasattr(obj, 'Value'):
            obj = obj.Value
        return obj

    try:
        # ESTRAIAMO IL TIMESTAMP ORIGINALE (TSN)
        # Usiamo il tempo in cui il dato è stato generato dal Publisher, non quando è arrivato.
        ts_source = None
        if hasattr(fields[0], 'Value') and hasattr(fields[0].Value, 'SourceTimestamp'):
            ts_source = fields[0].Value.SourceTimestamp

        # Nel caso dovesse mancare prendiamo il tempo attuale
        if ts_source is None:
            ts_source = datetime.now()

        # ESTRAIAMO I DATI (Ordine posizionale basato su create_meta_data)
        pos = float(get_val(fields[0]))
        temp = float(get_val(fields[1]))
        vib = float(get_val(fields[2]))
        volt = float(get_val(fields[3]))

        # --- PREPARAZIONE PUNTO INFLUX ---
        p = Point("misurazioni_fx") \
            .tag("source", "Advantech_Device") \
            .field("JointPosition", pos) \
            .field("Temperature", temp) \
            .field("Vibration", vib) \
            .field("Voltage", volt) \
            .time(ts_source)

        # --- SCRITTURA ASINCRONA (NON BLOCCANTE) ---

        self.write_api.write(bucket=INFLUX_BUCKET, org=INFLUX_ORG, record=p)

```

Figura 34 - Estrazione SourceTimestamp, decodifica dati e scrittura su DB (metodo della classe OnDataReceived)

L'interazione con InfluxDB è gestita tramite un client asincrono configurato con policy di Batching. I dati non vengono scritti singolarmente (operazione che bloccherebbe il loop di ricezione), ma accumulati in un buffer in memoria e inviati al database solo al raggiungimento di una soglia dimensionale (500 punti) o temporale (1000ms).

```

# Classe principale che gestisce la ricezione dei pacchetti OPC UA FX e il buffering verso InfluxDB.
class OnDataReceived:
    def __init__(self):
        # --- INIZIALIZZAZIONE CONNESSIONE DB ASINCRONA ---
        print(f"Connessione a InfluxDB ({INFLUX_URL}) in modalità ASYNC...")

        self.client = InfluxDBClient(url=INFLUX_URL, token=INFLUX_TOKEN, org=INFLUX_ORG)

        # 1. DEFINIAMO LE REGOLE DEL BATCHING
        # batch_size=500: Spedisci se hai accumulato 500 punti
        # flush_interval=1000: Spedisci comunque se è passato 1 secondo (1000ms)
        # retry_interval=5000: Se fallisce, riprova dopo 5 secondi
        write_options = WriteOptions(batch_size=500,
                                     flush_interval=1000,
                                     jitter_interval=0,
                                     retry_interval=5000)

        # 2. ISTANZIAMO LE CALLBACK
        callback = BatchingCallback()

        # 3. CREIAMO L'API ASINCRONA
        try:
            self.write_api = self.client.write_api(
                write_options=ASYNCHRONOUS,
                write_options_config=write_options,
                success_callback=callback.success,
                error_callback=callback.error,
                retry_callback=callback.error
            )
            print("Client Async Pronto (Batch: 500pt o 1s)")
        except Exception as e:
            print(f"Errore inizializzazione DB: {e}")

```

Figura 35 – Definizione regole di scrittura su DB

```

async def runner(): # loop principale
    print("In attesa di pacchetti (Scrittura Async attiva)...")
    async with ps:
        while True:
            await asyncio.sleep(1)

loop.run_until_complete(runner()) # avvio loop

```

Figura 36 - Loop in attesa di pacchetti UDP

3.5 Archiviazione dati e dashboard

Per completare l’ecosistema del Digital Twin di ASTRID è stata implementata una pipeline basata su InfluxDB per l’archiviazione e Grafana per la visualizzazione. A differenza dei moduli di controllo (Publisher/Subscriber), eseguiti in Bare Metal per

garantire l'accesso diretto ai timestamp hardware, lo stack di monitoraggio è stato dispiegato su Docker.

```
services:
  influxdb:
    image: influxdb:2.7.12
    container_name: influxdb
    restart: always
    # Rete Host: usa direttamente la scheda di rete dell'Orin
    network_mode: "host"
    environment:
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_USERNAME=admin
      - DOCKER_INFLUXDB_INIT_PASSWORD=password123
      - DOCKER_INFLUXDB_INIT_ORG=unibo
      - DOCKER_INFLUXDB_INIT_BUCKET=tesi_tsn
      - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=my-super-secret-token-auth-token
    volumes:
      - influxdb_data:/var/lib/influxdb2
      - influxdb_config:/etc/influxdb2

  grafana:
    image: grafana/grafana:10.4.0
    container_name: grafana
    restart: always
    network_mode: "host"
    depends_on:
      - influxdb
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=password
    volumes:
      - grafana_data:/var/lib/grafana
```

Figura 37 – Dockerfile

I servizi sul nodo Edge (Orin) sono gestiti tramite Docker Compose. Un'impostazione rilevante è la direttiva `network_mode`: "host" che rimuove l'isolamento tipico dei container permettendo al database e alla dashboard di condividere direttamente lo stack IP dell'host. Questo garantisce che il Subscriber locale possa scrivere su localhost:8086 senza l'overhead del NAT.

3.5.1 Persistenza dei dati: InfluxDB

Il database scelto per l'archiviazione è InfluxDB. Si tratta di un Time Series Database che a differenza dei tradizionali database relazionali (DBMS), progettati per gestire relazioni complesse tra entità statiche, è ottimizzato specificamente per l'inserimento ad alta velocità di dati indicizzati temporalmente. Ogni record non è una riga in una tabella generica, ma un Data Point immutabile costituito essenzialmente da tre elementi: un timestamp (chiave primaria), un set di Tags (metadati per l'indicizzazione) e i Fields (i valori misurati).^[16] Il database riceve i dati dal Subscriber e li organizza in questa struttura:

- Bucket: `tesi_tsn` (Contenitore logico dei dati).
- Measurement: `misurazioni_fx`.
- Tags: `source = "Advantech_Device"`.

- Field: JointPosition, Temperature, Vibration, Voltage.

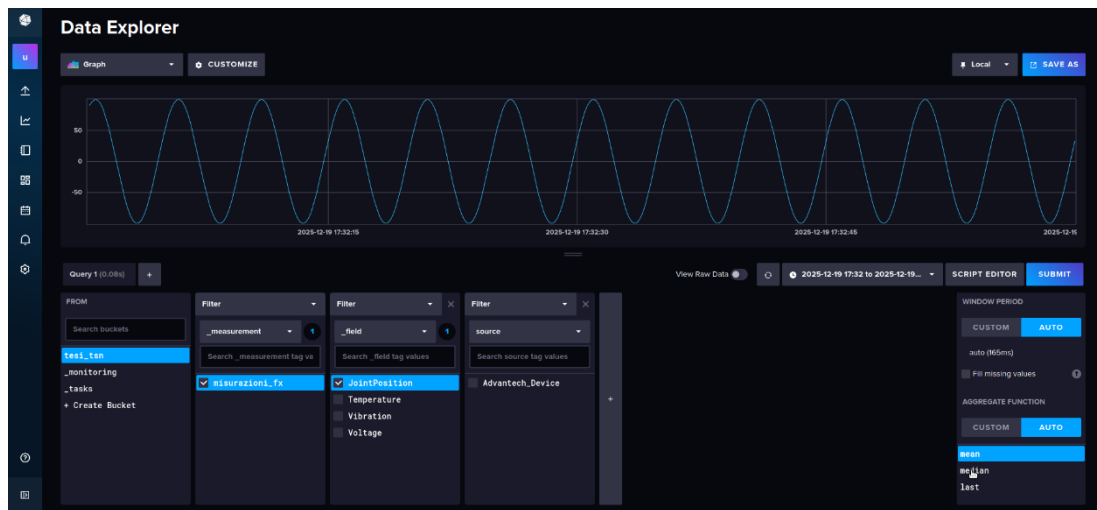


Figura 38 - Visualizzazione dati su InfluxDB

3.5.2 Dashboard Grafana

L'interfaccia utente del Digital Twin è stata realizzata con Grafana connesso direttamente ad InfluxDB. La dashboard permette di monitorare in tempo reale lo stato del macchinario emulato.

La dashboard è composta da diversi pannelli che mostrano le variabili simulate. I pannelli superiori mostrano l'ultimo valore ricevuto per le variabili come temperatura o voltaggio. Il pannello inferiore invece tiene traccia della variabile JointPosition. Il grafico mostra chiaramente la reattività del sistema, la parte destra del grafico indica un infittimento delle oscillazioni dell'onda sinusoidale (aumento della frequenza). Questo comportamento corrisponde all'istante in cui è stato inviato un comando di cambio velocità (Speed_Input) tramite il canale TCP. Il flusso dati UDP ha riportato fedelmente la nuova dinamica senza perdita di pacchetti.

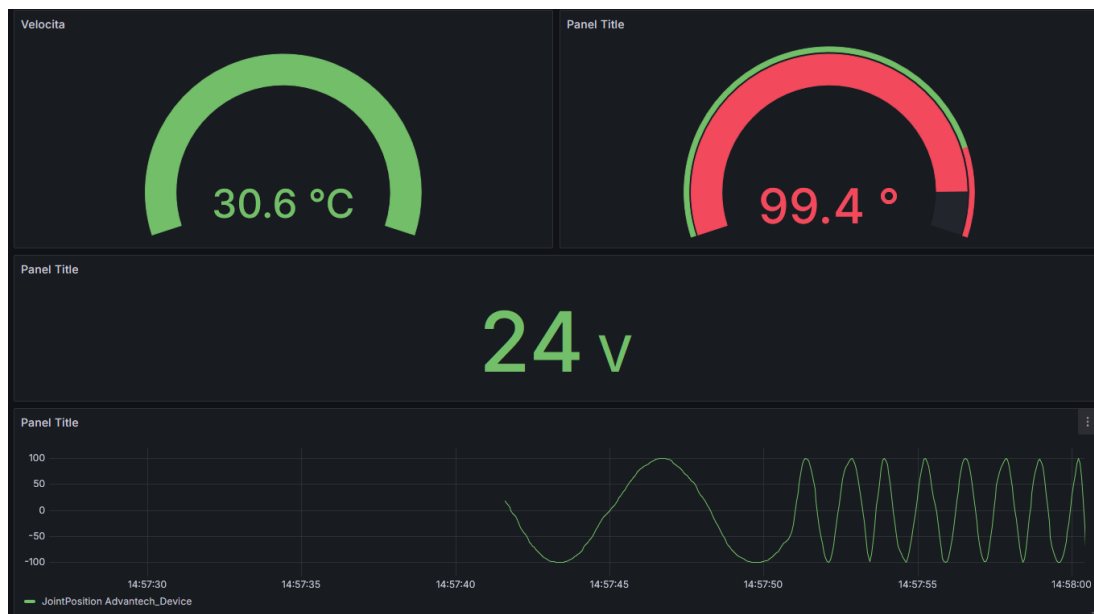


Figura 39 - Dashboard Grafana

Capitolo 4: Test e analisi dei risultati

4.1 Setup degli esperimenti

I test sperimentali per la validazione di ASTRID sono stati eseguiti sulla linea pilota del Competence Center Bi-Rex, un ambiente che replica le condizioni di una fabbrica intelligente. L'obiettivo è quantificare il miglioramento introdotto dal protocollo TSN rispetto a una rete Ethernet convenzionale all'interno dell'architettura proposta.

La piattaforma hardware di test su cui è stato dispiegato ASTRID è costituita dai seguenti nodi:

- Nodo Publisher: PC industriale Advantech (architettura x86)
- Nodo Subscriber: NVIDIA Jetson Orin (Architettura ARM)
- Switch Relyum

4.1.1 Metodologie di stress della rete

In una rete con un'ampia larghezza di banda il traffico di telemetria composto da pochi KB/s verrebbe trasmesso istantaneamente. Per validare l'efficienza dei meccanismi del protocollo TSN è quindi necessario simulare uno scenario di congestione critica. Per realizzare questo scenario Worst-Case si è agito su due fronti: la riduzione della capacità del canale e la saturazione del buffer.

Per prima cosa è stata limitata la banda dell'Orin tramite il tool `ethtool`. L'interfaccia di rete è stata forzata ad operare a 100Mbps creando un collo di bottiglia facile da saturare. Il comando utilizzato è stato:

```
sudo ethtool -s eth1 speed 100 duplex full autoneg off
```

Con il tool `Iperf3` è stato invece generato il rumore per saturare la rete. Il tool inietta un flusso di dati UDP Best Effort che consuma la quasi totalità della banda disponibile. Sul nodo publisher è stato utilizzato il comando:

```
iperf3 -c 192.168.37.100 -u -b 0 -t 100
```

L'opzione -b 0 dice al software di ignorare i limiti di banda e trasmettere pacchetti alla massima velocità permessa dalla CPU e dalla scheda di rete. Inoltre, per valutare il comportamento dell'infrastruttura al variare del traffico di disturbo sono state usate anche le opzioni: -b 200M, -b 500M e -b 900M.

4.1.2 Metodologie di test

Per valutare le performance del protocollo TSN in una rete industriale sono state eseguite diverse tipologie di test in scenari operativi differenti.

Per prima cosa è stata valutata la latenza della rete, cioè il tempo che un pacchetto impiega per andare dal mittente al destinatario. I primi test sono stati eseguiti in una situazione di rete scarica. Il traffico di disturbo è stato poi gradualmente aumentato fino a raggiungere la saturazione del canale simulando un Worst-Case. L'analisi della rete in questa fase è stata effettuata utilizzando tcpdump.

Replicando la stessa metodologia di stress della rete è stata eseguita anche la misurazione del jitter, confrontando il comportamento della rete standard rispetto a quella con TSN abilitato.

Il jitter viene calcolato come la latenza tra pacchetti successivi. In questo caso stiamo misurando quanto la latenza oscilla tra un pacchetto e un altro, ovvero l'oscillazione del ritardo di consegna. Ad esempio, se un nodo publisher pubblica un pacchetto ogni 1ms il jitter risulta nullo se il subscriber riceve un pacchetto ogni 1ms. In una rete reale però, a causa del traffico sulla rete, un pacchetto può impiegare più tempo. Magari passano 5 millisecondi senza che arrivi nulla e poi improvvisamente arrivano 5 pacchetti tutti insieme in un istante. Il jitter è importante in ambito industriale perché se un dato impiega sempre lo stesso tempo per essere consegnato il sistema è deterministico ed è possibile calibrarlo su quel ritardo noto. Al contrario, se i tempi di consegna dei comandi oscillano in modo imprevedibile in un range variabile (es. tra 1 e 5 ms), i sistemi di controllo perdono la sincronizzazione, generando anomalie e innescando blocchi di sicurezza. Il protocollo TSN ha proprio l'obiettivo di abbattere il jitter.

L'ultima analisi è stata quella sul livello applicativo. A questo scopo è stato sviluppato un programma Python in grado di calcolare la latenza dei messaggi di OPC UA FX. Questa analisi ha permesso di valutare i ritardi prodotti dallo stack software di ASTRID e dal sistema operativo.

4.2 Analisi della latenza

In questa fase di test l'obiettivo è misurare il tempo di transito dei pacchetti confrontando le prestazioni dell'infrastruttura in assenza e in presenza dei meccanismi di TSN al variare del carico di rete (0, 200, 500 e 900 Mbps).

4.2.1 Acquisizione dei dati

I dati sono stati catturati ai capi della comunicazione, cioè all'interno del nodo Publisher (Advantech) e del nodo Subscriber (Orin). A questo scopo è stato utilizzato il tool tcpdump con la massima precisione temporale possibile tramite l'opzione: "--time-stamp-precision=nano".

Tutte le catture sono state salvate in file .pcap e sono state eseguite in maniera differente in base allo scenario:

- Nello scenario TSN è stato filtrato il traffico appartenente alla VLAN 100: "sudo tcpdump -i enp4s0 vlan 100 and host 192.168.99.2 -w tx_tsn.pcap --time-stamp-precision=nano".
- Nello scenario senza TSN è stato invece catturato il traffico UDP che transitava sulla porta 4840: "sudo tcpdump -i enp4s0 udp port 4840 and host 192.168.37.100 -w tx_notsn.pcap --time-stamp-precision=nano"

4.2.2 Calcolo della latenza

Per calcolare la latenza è stato sviluppato un software Python con lo scopo di eseguire il matching dei pacchetti tra i file di cattura in trasmissione e in ricezione.

Per prima cosa il programma esegue il parsing degli header dai file .pcap, estraendo per ogni pacchetto il timestamp di cattura. Estrae poi i primi 32 byte dal payload di ogni pacchetto. Questa porzione di dati contiene l'intestazione dinamica del messaggio OPC UA che include il numero di sequenza e il timestamp di generazione alla sorgente. Poiché il publisher genera un pacchetto ogni millisecondo, il Sequence Number e il timestamp cambiano in ogni singolo pacchetto ed è quindi possibile usarli come firma. A questo punto, l'algoritmo associa ogni pacchetto trasmesso al corrispondente pacchetto ricevuto cercando la stessa firma. Infine, la latenza viene calcolata come la differenza tra il timestamp di arrivo e quello di partenza convertendo poi il risultato in millisecondi.

4.2.3 Analisi dei risultati

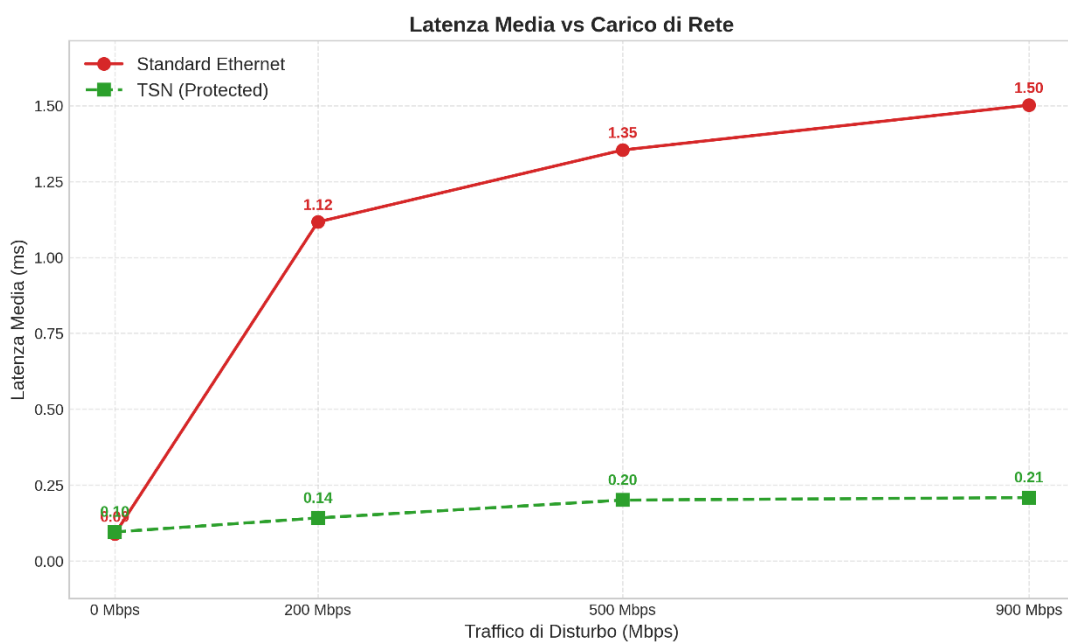


Figura 40 - Latenza media vs carico di rete

Come si può vedere dal grafico relativo alla latenza media in funzione del traffico di disturbo, in assenza di carico (0 Mbps) le prestazioni delle due configurazioni sono praticamente identiche. All'aumentare del traffico in background però la rete standard

subisce un drastico degrado, mentre quella che utilizza TSN mantiene una latenza stabile anche in condizioni di massimo stress.

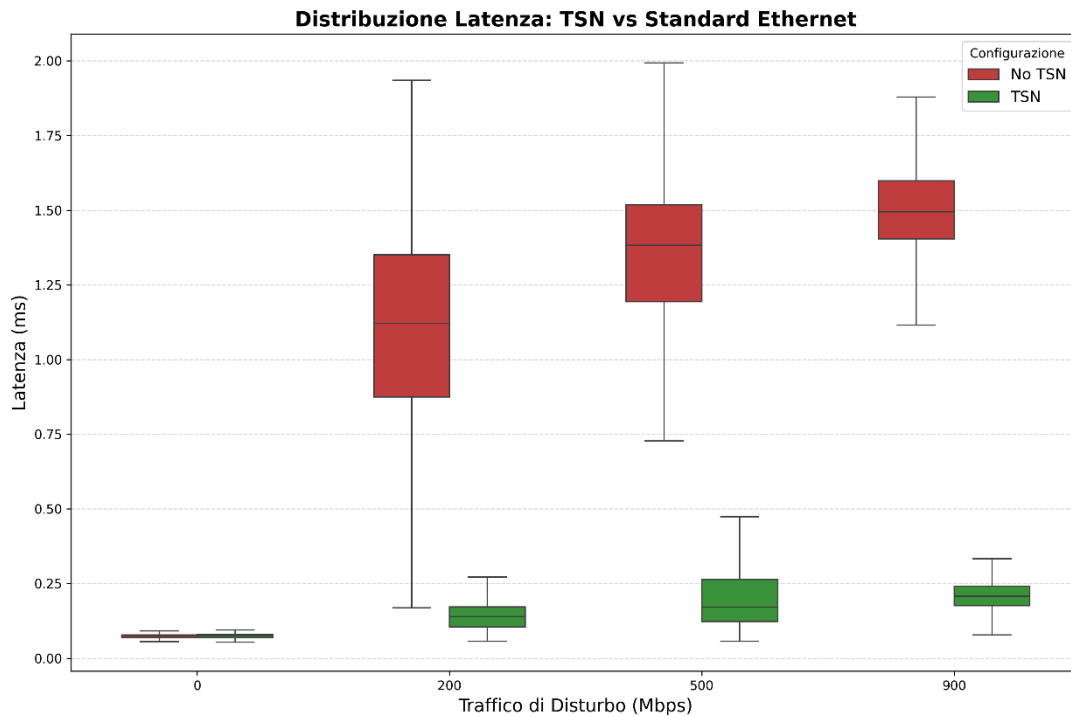


Figura 41 - Boxplot latenza

Osservando il boxplot è possibile notare che TSN presenta riquadri (box) molto schiacciati. Questo indica che la maggior parte dei pacchetti ha una latenza quasi identica, garantendo un forte determinismo. Al contrario, la rete standard ha box molto più alti che denotano una maggiore instabilità. I lunghi baffi della configurazione non deterministica, inoltre, evidenziano come una parte dei pacchetti subisca un forte ritardo a causa del traffico di disturbo. I baffi corti della configurazione TSN confermano invece che anche i casi peggiori (Worst-Case) rimangono entro limiti temporali ristretti e accettabili. L'analisi delle mediane sottolinea come la latenza subisca un forte incremento nello scenario standard all'aumentare del carico, mentre nell'architettura TSN resta quasi costante.

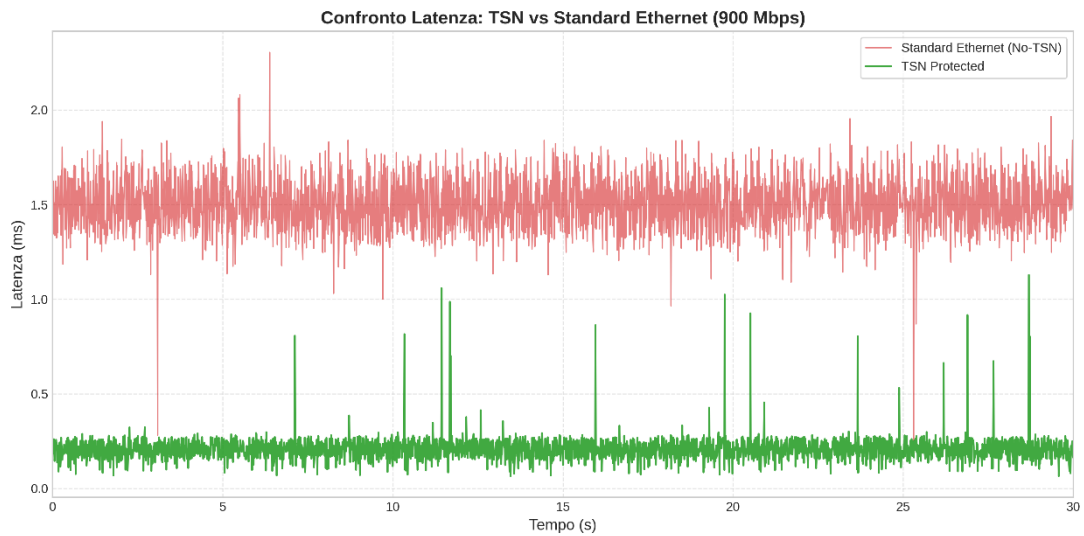


Figura 42 - Confronto latenza TSN e Ethernet Standard (disturbo 900 Mbps)

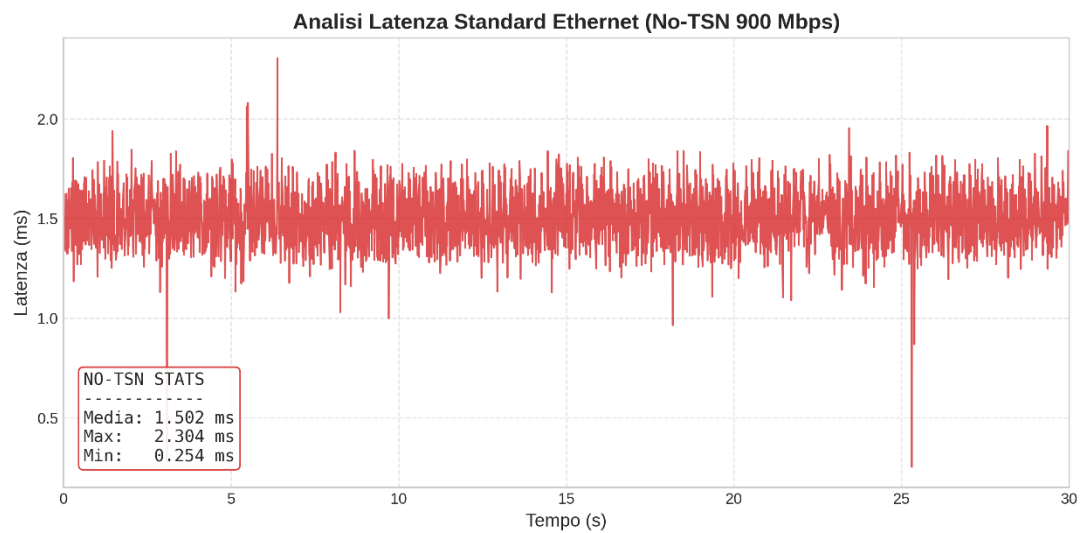


Figura 43 - Latenza no TSN (disturbo 900 Mbps)

L'analisi dello scenario limite a 900 Mbps mostra il comportamento della rete in un intervallo di 30 secondi. La traccia relativa all'Ethernet standard (no TSN) evidenzia ripetute fluttuazioni dovute all'interferenza del traffico di disturbo.

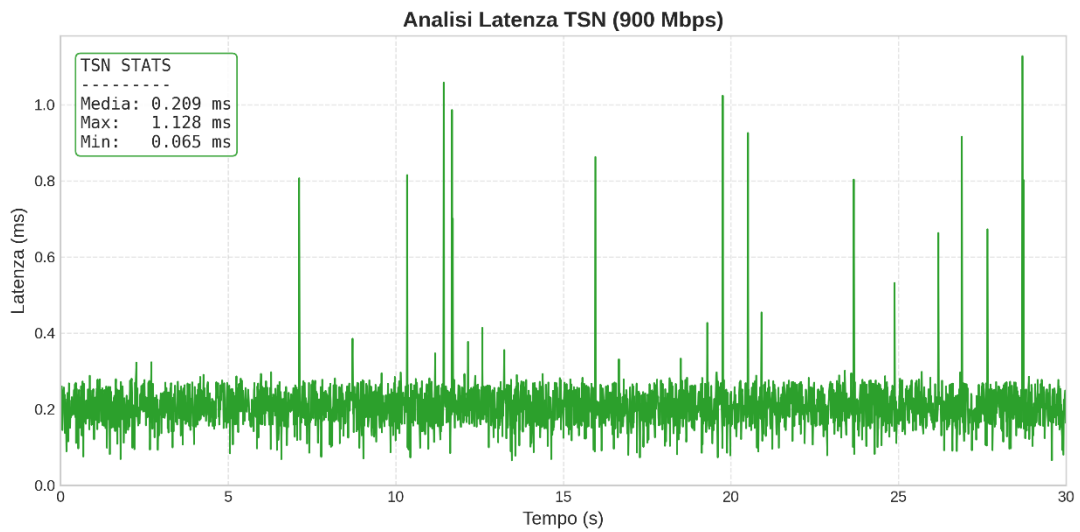


Figura 44 - Latenza TSN (disturbo 900 Mbps)

Con l'implementazione di TSN la latenza è confinata in una fascia molto più ristretta e molto vicina allo zero (media di 0.209 ms). Nel grafico è anche possibile vedere dei picchi causati dal TAS. Visto che l'applicativo Python sul nodo Publisher opera in regime Soft Real-Time e non è sincronizzato a livello hardware con lo switch può succedere che un pacchetto venga inviato durante i 600 microsecondi in cui il gate per la coda prioritaria Q6 è chiuso. Lo switch riceve quindi il pacchetto ma lo trattiene nel buffer. Il pacchetto sarà poi inviato nel ciclo successivo appena il gate della coda Q6 si aprirà. Poiché il ciclo ha una durata fissa di 1 millisecondo il ritardo massimo è comunque limitato ed inferiore ai valori medi e minimi registrati nell'Ethernet standard.

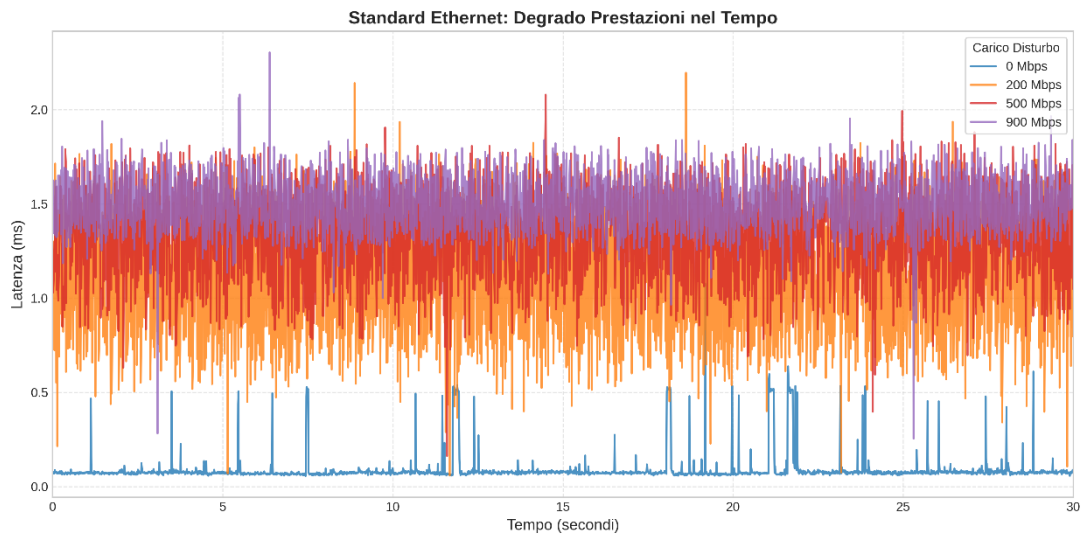


Figura 45 - Sovrapposizione scenari di carico no TSN

La sovrapposizione dei vari scenari di carico sottolinea ancora di più il comportamento molto differente della rete. Nell’Ethernet standard il traffico di background trasla la banda di oscillazione su valori di ritardo molto alti. In TSN, invece, le tracce sono quasi indistinguibili e sovrapposte. Questa comparazione dimostra come l’isolamento del traffico renda i pacchetti critici immuni alla dinamica del traffico in background.

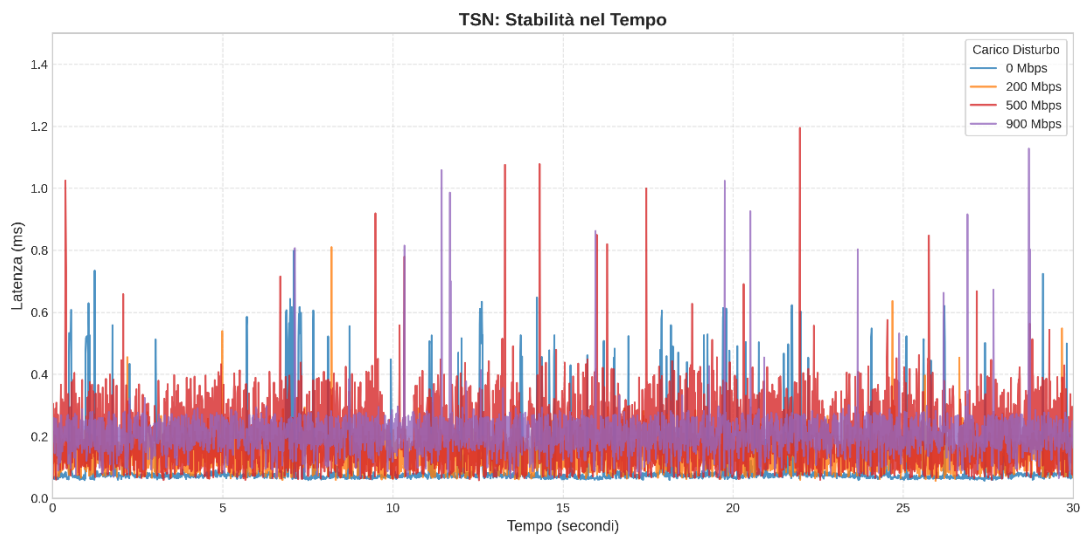


Figura 46 - Sovrapposizione scenari di carico TSN

4.3 Analisi del jitter

In questa fase l'obiettivo è valutare la stabilità del flusso dati misurando il jitter della rete in presenza e assenza dei meccanismi di TSN e al variare del carico.

4.3.1 Metodologia di misurazione

A differenza della latenza calcolata nel paragrafo precedente, in cui è stato misurato il tempo di transito assoluto dal nodo Publisher al nodo Subscriber, il jitter è stato calcolato analizzando esclusivamente il traffico in ricezione.

Per effettuare la misurazione, l'acquisizione dei dati è stata eseguita tramite il tool `tcpdump` in ascolto sull'interfaccia del Subscriber. Il comando eseguito è stato:

```
sudo tcpdump -i eth1 -n -ttt "udp port 4840"
```

L'opzione `-ttt` indica al programma di calcolare e stampare direttamente il delta time, cioè la differenza temporale tra la cattura del pacchetto corrente e quella del pacchetto precedente.

Poiché il nodo Publisher genera un messaggio con una periodicità nota di 1ms, il calcolo del jitter si ottiene sottraendo il periodo atteso al delta time misurato da `tcpdump`. Un risultato vicino allo zero indica un flusso dati prevedibile, mentre valori elevati o forti fluttuazioni indicano un arrivo a singhiozzo.

4.3.2 Analisi dei risultati

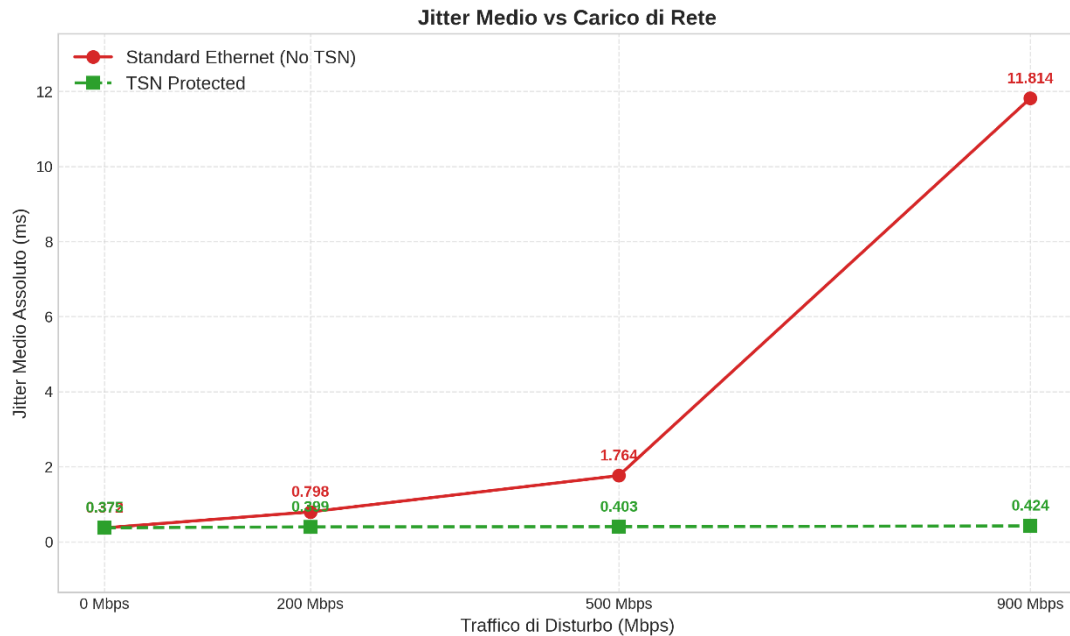


Figura 47 - Jitter medio VS carico di rete

Come illustrato nel grafico relativo al jitter medio, in assenza di traffico di background le due reti hanno prestazioni quasi identiche (circa 0.37 ms). All'aumentare della congestione le due configurazioni divergono in maniera considerevole. La rete TSN mantiene un jitter molto stabile, crescendo da 0.37 a 0.42. Al contrario, il traffico Ethernet standard subisce un forte degrado raggiungendo un jitter di ben 11.81 ms con un disturbo di 900 Mbps.

È importante evidenziare la differenza tra questo valore e quello della latenza media misurata in precedenza (circa 1.50 ms). Nonostante il tempo di latenza medio non sia eccessivo, anche in mancanza dei meccanismi di TSN, il jitter così alto sottolinea che il flusso dei dati non è per nulla deterministico. I pacchetti, accumulandosi nei buffer saturi degli switch, perdono totalmente la loro cadenza temporale. Se un sistema robotico si aspetta un comando di controllo ogni millisecondo, un'interferenza che sfasa l'arrivo dei dati di 11 ms rende la comunicazione inutilizzabile, causando inevitabilmente errori o blocchi del macchinario.

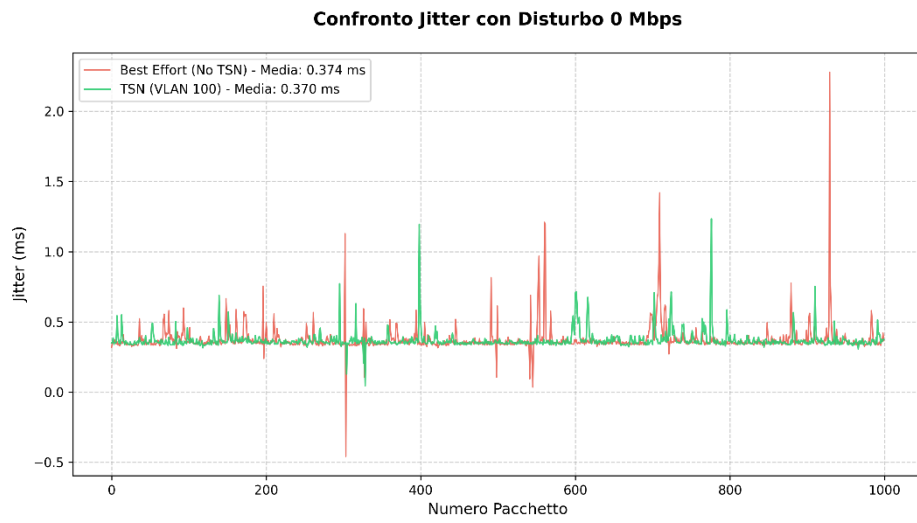


Figura 48 - Jitter con disturbo 0 Mbps

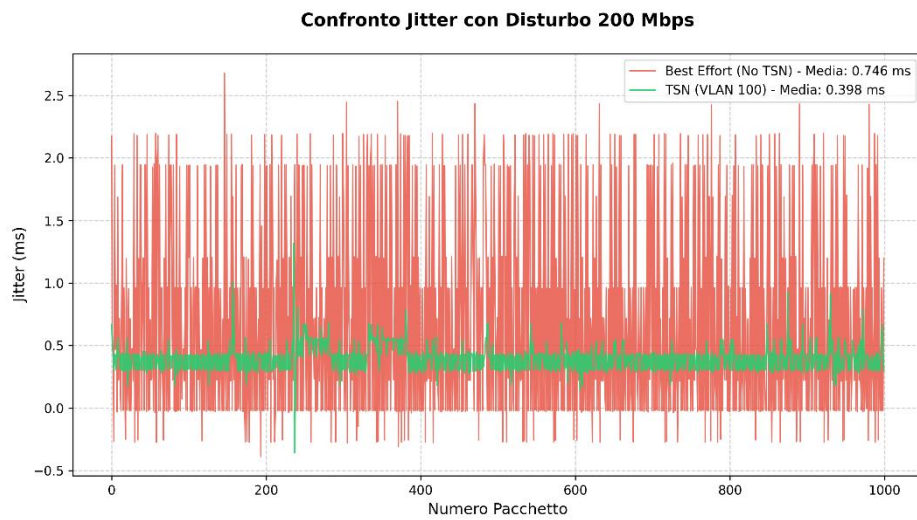


Figura 49 - Jitter con disturbo 200 Mbps

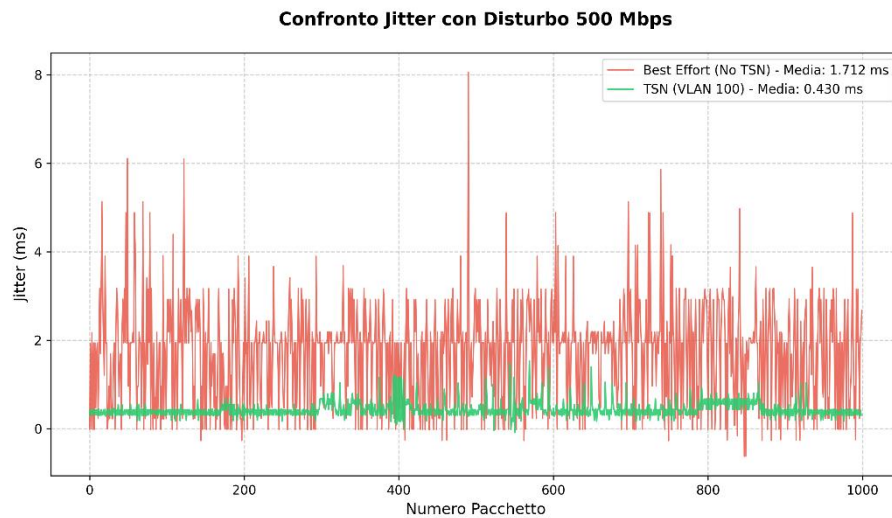


Figura 50 - Jitter con disturbo 500 Mbps

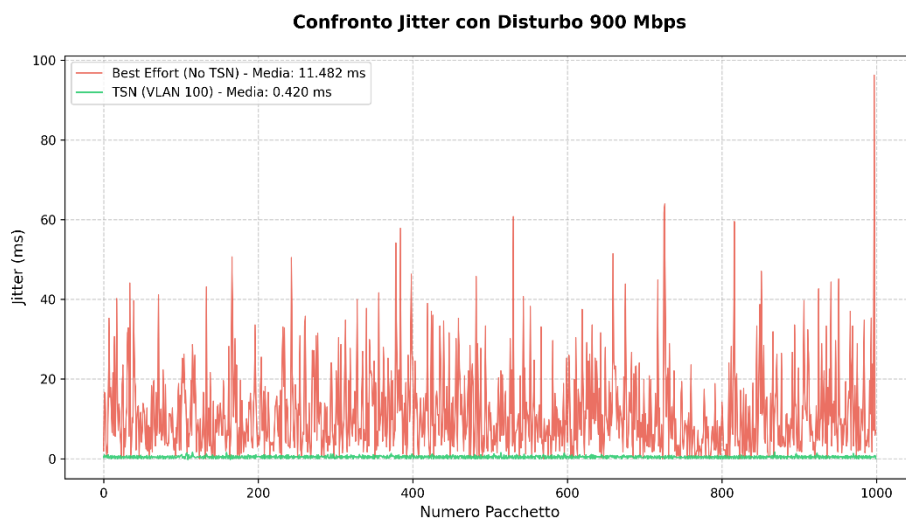


Figura 51 - Jitter con disturbo 900 Mbps

Osservando i grafici temporali che mostrano la variazione del jitter rispetto alla variazione del traffico di disturbo è possibile osservare il comportamento della rete sotto stress. Nelle tracce rosse relative all'Ethernet standard si osservano dei picchi estremi con valori che superano i 60 o addirittura gli 80 ms. Questi valori mostrano come i pacchetti non viaggiano lentamente, ma si accumulano generando un blackout delle comunicazioni. In assenza dei meccanismi di protezione di TSN, gli switch in saturazione scartano i pacchetti OPC UA per fare spazio al traffico di disturbo. La rete

rimane quindi muta per diversi millisecondi. I picchi rappresentano proprio il momento in cui il traffico torna a scorrere. Visto che il Publisher invia un pacchetto ogni millisecondo, un picco di 80ms di jitter equivale a circa 80 pacchetti di controllo trattenuti o scartati dalla rete. Oltre ai picchi, abbiamo anche dei crolli evidenti nel grafico dovuti alle raffiche (burst). Quando gli switch non sono più congestionati svuotano improvvisamente il loro buffer inviando in rapida successione i pacchetti accumulati. Il nodo Subscriber riceve quindi pacchetti che hanno un delta time quasi nullo. La traccia verde (TSN) è invece in netto contrasto e appare quasi piatta. Le oscillazioni sono legate al ciclo di apertura del gate della coda prioritaria Q6.

4.4 Analisi dell'overhead dello stack software

Nelle applicazioni industriali Real-Time, il ritardo di comunicazione non dipende soltanto dal tempo di transito dei pacchetti sul cavo, ma anche dai tempi di elaborazione dei nodi. In questa analisi l'obiettivo è quantificare l'impatto che ha lo stack software (sistema operativo, driver di rete, librerie applicative) sulle prestazioni.

4.4.1 Metodologie di misurazione

Per isolare il tempo di elaborazione software dal tempo di trasmissione dei pacchetti, il test è stato condotto con una doppia misurazione in parallelo eseguita in uno scenario di rete congestionata (oltre 900 Mbps). Per questo scopo, viene utilizzato il nodo Publisher di ASTRID scritto in Python che all'interno dei messaggi inserisce il timestamp esatto di generazione alla sorgente.

La latenza di rete (Hardware-Level) è stata misurata ai capi della comunicazione tramite tcpdump. I file .pcap ottenuti sono poi stati elaborati tramite un altro script Python che esegue il matching dei pacchetti tra trasmissione e ricezione estraendo la firma univoca dal payload e calcolando la latenza a livello fisico.

La latenza applicativa (Software-Level) viene misurata direttamente all'interno del codice Python del Subscriber. Il tempo di arrivo viene registrato nel momento esatto

in cui la libreria OPC UA decodifica il pacchetto UDP e rende il dato disponibile alla logica applicativa. La latenza totale viene quindi calcolata sottraendo a questo valore il TimestampTX originario estratto dal payload.

I dati raccolti dai due metodi vengono quindi sincronizzati per calcolare la differenza ed estrarre l'overhead dello strato software.

4.4.2 Analisi dei risultati

I risultati ottenuti vengono mostrati tramite stacked area chart che mostrano la scomposizione della latenza totale nei suoi due componenti principali.

4.4.2.1 Scenario TSN

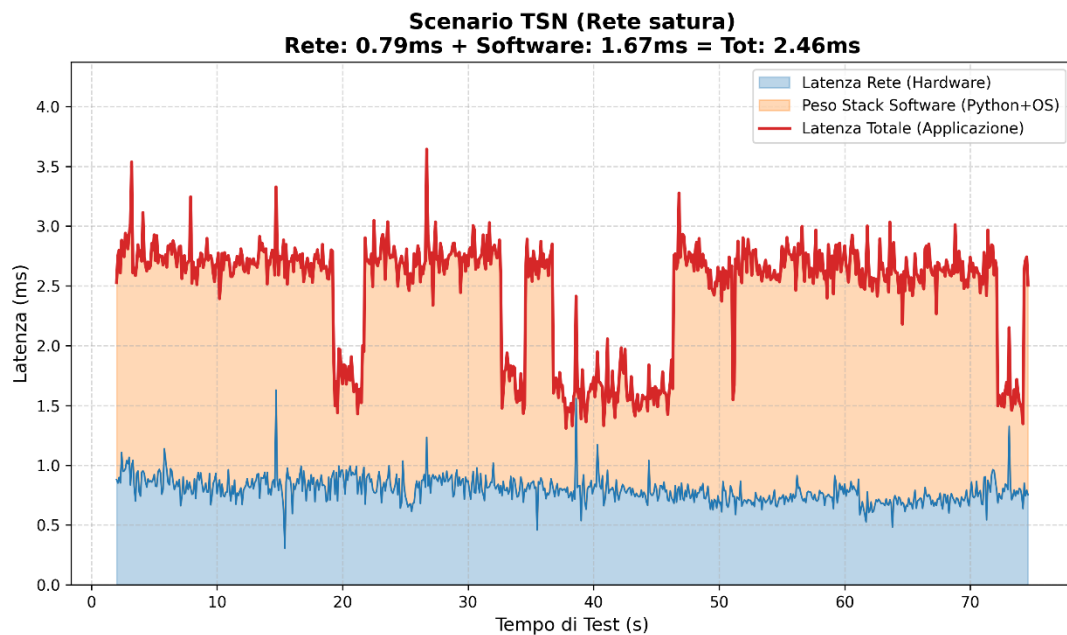


Figura 52 - Overhead stack software TSN

Osservando il grafico relativo allo scenario con TSN possiamo notare come la rete hardware sia estremamente reattiva e stabile (latenza media 0.79 ms). L'elaborazione

software risulta molto più incisiva aggiungendo un ritardo di 1.67 ms. Il tempo di transito totale di un pacchetto percepito dall'applicazione si aggira quindi intorno ai 2.46 ms. Altro aspetto che è possibile notare è che la traccia azzurra è piatta e deterministica mentre l'area rossa introduce una forte incertezza dovuta alle ampie fluttuazioni. Questo dimostra che, sebbene il TSN garantisca il determinismo sul cavo, l'uso di un sistema operativo non Real-Time e di un linguaggio interpretato come Python introduce un'inevitabile instabilità nell'elaborazione del nodo.

4.4.2.2 Scenario Ethernet standard

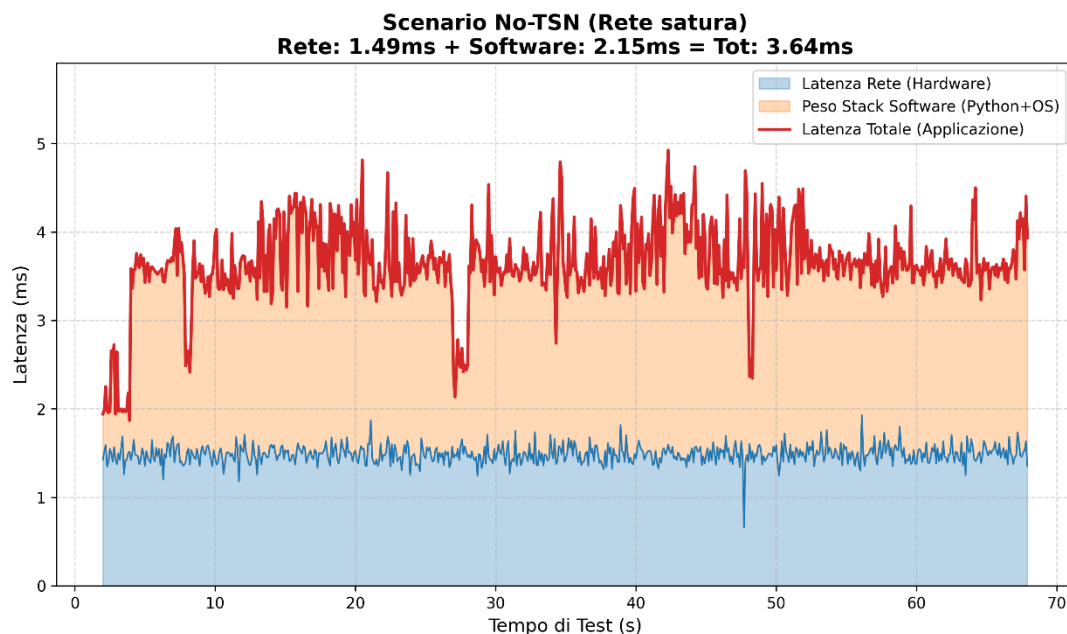


Figura 53 - Overhead stack software no TSN

Il grafico relativo allo scenario best-effort evidenzia un degrado su entrambi i fronti. La latenza di rete subisce un aumento a causa dell'interferenza del traffico di disturbo arrivando a una media di 1.49ms. Anche lo stack software mostra una sofferenza maggiore raggiungendo una media di ritardo di 2.15ms. È possibile notare come le fluttuazioni della rete vengano amplificate dallo stack software.

4.4.2.3 Analisi conclusiva

L'analisi di questo test parallelo permette di trarre una conclusione sulla progettazione di architetture industriali: l'utilizzo del protocollo TSN risolve il problema della congestione del traffico fisico offrendo un canale di comunicazione rapido e privo di jitter. Tuttavia, affinché un'applicazione industriale funzioni al meglio è necessario accoppiare al determinismo della rete anche un determinismo di elaborazione. In questo scenario il collo di bottiglia temporale si sposta dai cavi di rete alla CPU rendendo necessario adottare altre strategie come sistemi operativi Real-Time e stack software compilati a basso livello come C/C++.

4.5 Monitoraggio e Visualizzazione Real-Time tramite Grafana

Oltre l'analisi effettuata tramite tcpdump e script Python, è stata implementata anche una soluzione di monitoraggio in tempo reale in modo da verificare il comportamento del Digital Twin di ASTRID anche durante l'esecuzione. L'architettura prevede l'utilizzo di InfluxDB come database e di Grafana come interfaccia di visualizzazione.

4.5.1 Configurazione della dashboard



Figura 54 - Dashboard Grafana monitoraggio

La dashboard permette di visualizzare le metriche di rete (latenza e jitter) e i dati di telemetria (posizione del giunto, vibrazioni). Questa visualizzazione è molto utile per capire se un'anomalia nella rete sta degradando la qualità del controllo del macchinario.

La dashboard è divisa in quattro pannelli che eseguono query specifiche scritte nel linguaggio Flux.

4.5.1.1 Analisi della latenza

Il pannello superiore a sinistra mostra il valore medio della latenza in modo da mostrare immediatamente lo stato di salute della connessione. Il calcolo della Latenza Media (visualizzato come 2.06 ms) viene effettuato calcolando la media di tutti i campioni registrati nel periodo selezionato.

```

from(bucket: "tesi_tsn")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_measurement"] == "misurazioni_fx")
  |> filter(fn: (r) => r["_field"] == "Latency_ms")
  |> mean()
  |> yield(name: "Latenza Media")

```

Figura 55 - Query analisi latenza

Il grafico sottostante traccia invece l'andamento della latenza per ogni istante. In questo modo è possibile identificare i vari picchi.

```

from(bucket: "tesi_tsn")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_measurement"] == "misurazioni_fx")
  |> filter(fn: (r) => r["_field"] == "Latency_ms")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
  |> yield(name: "Latenza Media")

```

Figura 56 - Query traccia latenza

4.5.1.2 Calcolo del Jitter

Il jitter è stato calcolato matematicamente attraverso la deviazione standard della latenza all'interno di una finestra temporale di 1 secondo.

```

from(bucket: "tesi_tsn")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_measurement"] == "misurazioni_fx")
  |> filter(fn: (r) => r["_field"] == "Latency_ms")
  |> aggregateWindow(every: 1s, fn: stddev, createEmpty: false)
  |> mean()
  |> yield(name: "Jitter Medio")

```

Figura 57 - Query analisi jitter

4.5.1.3 Dati di telemetria

La parte inferiore della dashboard visualizza i dati trasportati dal payload OPC UA. Il grafico sinusoidale rappresenta il movimento emulato del giunto. La fluidità e la continuità dell'onda sinusoidale sono indicatori visivi della corretta ricezione dei pacchetti. Eventuali discontinuità nell'onda denoterebbero perdita di pacchetti. Inoltre, è anche possibile visionare i dati della vibrazione del macchinario.

```
from(bucket: "tesi_tsn")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r["_measurement"] == "misurazioni_fx")
  |> filter(fn: (r) => r["_field"] == "JointPosition")
  |> aggregateWindow(every: v.windowPeriod, fn: mean, createEmpty: false)
  |> yield(name: "Posizione")
```

Figura 58 - Grafana Joint Position query

Conclusioni e Sviluppi Futuri

Sintesi delle attività e obiettivi raggiunti

In questo elaborato ho dimostrato la fattibilità e l'efficienza di ASTRID, un'architettura di rete deterministica basata sull'integrazione del protocollo OPC UA Field eXchange (FX) su un'infrastruttura Time Sensitive Networking (TSN). Attraverso la progettazione e lo sviluppo dell'architettura ibrida alla base di ASTRID, distribuita su nodi hardware eterogenei, è stato possibile garantire l'isolamento del traffico critico rispetto al traffico Best-Effort di disturbo.

I risultati sperimentali sono stati condotti sulla linea pilota di Bi-Rex su cui è stato emulato uno scenario di congestione critica (fino a 900 Mbps di traffico interferente). I dati ottenuti hanno dimostrato la superiorità dell'ecosistema TSN e l'affidabilità di ASTRID. Nonostante lo stack applicativo sviluppato in Python operi in un regime di Soft-RealTime, l'infrastruttura di rete configurata con gli standard IEEE 802.1Qbv (TAS) e IEEE 802.1AS (gPTP) ha garantito ottimi risultati. Mentre in Ethernet standard la latenza media è stata di 1.50 ms con picchi di jitter di 11.81 ms, la configurazione con TSN ha registrato una latenza media di 0.20 ms e un jitter stabile a 0.42 ms.

Il monitoraggio in tempo reale del Digital Twin è stato realizzato attraverso lo sviluppo di una pipeline dedicata. I dati sono stati scritti direttamente dal Subscriber Python in un Time Series Database (InfluxDB) e la dashboard di Grafana ha permesso la visualizzazione dei grafici e delle statistiche.

Analisi dei limiti implementativi

I test sperimentali hanno evidenziato che l'infrastruttura hardware e di rete ha risposto in maniera ottimale ai requisiti di determinismo, spostando il collo di bottiglia dalla trasmissione di rete all'elaborazione software. A fronte di una latenza hardware di soli 0.79 ms, lo stack software ha introdotto un ritardo medio di 1.67 ms.

Questo comportamento è imputabile alle scelte tecnologiche adottate per lo sviluppo degli applicativi di ASTRID. L'utilizzo del linguaggio Python e della libreria `opcua-asyncio` ha garantito vantaggi in termini di astrazione e di agilità della prototipazione. Operando in user space l'applicativo affronta i limiti del linguaggio: lo scheduling asincrono dell'event loop a causa della sleep, la gestione imprevedibile della memoria affidata al Garbage Collector e l'overhead dell'interprete. Questi elementi introducono un jitter software elevato.

Operando ad alto livello, inoltre, non è possibile interfacciarsi né direttamente con il tempo hardware di PTP né con il tagging a livello 2. È stato quindi necessario trovare soluzioni alternative come affidare il tagging VLAN (PCP 6) a regole di iptables e all'uso della tabella mangle aggiungendo quindi un ulteriore passaggio intermedio.

Sviluppi futuri

Il primo sviluppo possibile, alla luce dei risultati, è la riscrittura dei nodi Publisher e Subscriber di ASTRID con un linguaggio compilato come C o C++. L'obiettivo primario è eliminare l'overhead dell'interprete e ottenere maggior controllo sulle risorse del sistema. In questo scenario è possibile utilizzare la libreria `open62541`. Operando a un livello di astrazione più basso gli applicativi possono interfacciarsi direttamente con le socket di rete del Kernel Linux. In un'applicazione C è possibile operare con la socket option `SO_TXTIME`. Questa permette di assegnare ad ogni pacchetto UDP un timestamp esatto al nanosecondo e ricavato direttamente dal PTP Hardware Clock della scheda di rete. Grazie a questo approccio sarebbe possibile utilizzare le queuing discipline (qdisc) `etf` di Linux. L'algoritmo `etf` comanderebbe la scheda di rete ad eseguire l'invio del pacchetto nell'esatto microsecondo di apertura del gate configurato nello switch tramite il protocollo TAS. Questa configurazione renderebbe inutile l'utilizzo delle iptables e in fase di test restituirebbe probabilmente dei risultati ancora migliori rispetto a quelli mostrati nei test precedenti visto il minor overhead.

Indice delle figure

Figura 1 - Evoluzione industriale.....	6
Figura 2 - Cloud Continuum	8
Figura 3 - Ciclo di vita del prodotto	9
Figura 4 - Classificazione su flusso dati	11
Figura 5 - Meccanismo di accodamento	13
Figura 6 - Traditional IT/OT VS Converged Infrastructure	14
Figura 7 - Piramide dell'automazione	16
Figura 8 – Publisher.....	17
Figura 9 - Subscriber.....	17
Figura 10 - OPC UA FX message layer	19
Figura 11 - Differenza tra container e VM	21
Figura 12 – Logo Docker.....	21
Figura 13 - Schema IEEE 802.1AS	24
Figura 14 - NTP VS PTP	25
Figura 15 - Schema IEEE 802.1Qbv	26
Figura 16 - Topologia fisica.....	28
Figura 17 – Topologia logica.....	29
Figura 18 - Vlan: in alto Advantech, in basso Orin	30
Figura 19 - Vlan configuration	30
Figura 20 - Configurazione TAS	31
Figura 21 - Tabella mangle	32
Figura 22 - Tcpdump priorità vlan.....	32
Figura 23 - gptp.cfg	33
Figura 24 - PTP Hierachy	33
Figura 25 - ptp4l	34
Figura 26 - phc2sys.....	35
Figura 27 - Struttura dati visibile su UaExpert	36
Figura 28 - UaExpert	36
Figura 29 - Codice server TCP	37
Figura 30 – Loop e calcolo fisica simulata	38
Figura 31 – Definizione metadati UADP	39
Figura 32 - Filtro dati ingresso controllando WriterGroupId, DataSetWriter e PublisherId	40
Figura 33 - Metadati UADP.....	40
Figura 34 - Estrazione SourceTimestamp, decodifica dati e scrittura su DB (metodo della classe OnDataReceived)	41
Figura 35 – Definizione regole di scrittura su DB.....	42
Figura 36 - Loop in attesa di pacchetti UDP	42
Figura 37 – Dockerfile	43
Figura 38 - Visualizzazione dati su InfluxDB	44
Figura 39 - Dashboard Grafana	45
Figura 40 - Latenza media vs carico di rete.....	49

Figura 41 - Boxplot latenza	50
Figura 42 - Confronto latenza TSN e Ethernet Standard (disturbo 900 Mbps).....	51
Figura 43 - Latenza no TSN (disturbo 900 Mbps).....	51
Figura 44 - Latenza TSN (disturbo 900 Mbps).....	52
Figura 45 - Sovrapposizione scenari di carico no TSN	53
Figura 46 - Sovrapposizione scenari di carico TSN	53
Figura 47 - Jitter medio VS carico di rete.....	55
Figura 48 - Jitter con disturbo 0 Mbps.....	56
Figura 49 - Jitter con disturbo 200 Mbps.....	56
Figura 50 - Jitter con disturbo 500 Mbps.....	57
Figura 51 - Jitter con disturbo 900 Mbps.....	57
Figura 52 - Overhead stack software TSN.....	59
Figura 53 - Overhead stack software no TSN.....	60
Figura 54 - Dashboard Grafana monitoraggio	62
Figura 55 - Query analisi latenza	63
Figura 56 - Query traccia latenza.....	63
Figura 57 - Query analisi jitter.....	63
Figura 58 - Grafana Joint Position query	64

Bibliografia

[1] “Industria 4.0: storia, significato ed evoluzioni tecnologiche a vantaggio del business” [Online] Available: <https://www.digital4.biz/executive/industria-40-storia-significato-ed-evoluzioni-tecnologiche-a-vantaggio-del-business>

[2] “Cos'è l'industria 4.0?” [Online] Available: <https://www.sap.com/italy/products/scm/industry-4-0/what-is-industry-4-0.html>

[3] Commissione europea, Direzione generale per la ricerca e l'innovazione, *Industria 5.0: verso un'industria europea sostenibile, incentrata sull'uomo e resiliente*, Ufficio delle pubblicazioni dell'Unione europea, 2021, <https://data.europa.eu/doi/10.2777/308407>

[4] “Cloud Continuum: The Definition” [Online] Available: https://www.researchgate.net/publication/366277900_Cloud_Continuum_The_Definition

[5] “Che cos'è un gemello digitale?” [Online] Available: <https://www.ibm.com/it-it/think/topics/digital-twin>

[6] “Digital twin” [Online] Available: <https://www.sw.siemens.com/it-IT/technology/digital-twin/>

[7] “Digital twin: cos'è, come funziona e vantaggi del modello del gemello digitale” [Online] Available: <https://www.ai4business.it/intelligenza-artificiale/digital-twin-cos-e-come-funziona-il-modello-del-gemello-digitale/>

- [8] *“Ethernet-Based Real-Time and Industrial Communications”* [Online]
Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1435741>
- [9] *“Time-Sensitive Networking (TSN) – Il momento di agire è ora”* [Online]
Available: https://www.cc-link.org/files/cclinkie-tsn_whitepater_1_ita.pdf
- [10] *“OPC Unified Architecture Part 14: PubSub” Capitolo 5* [Online] Available:
<https://reference.opcfoundation.org/Core/Part14/v104/docs/>
- [11] *“OPC Unified Architecture Part 14: PubSub” Capitolo 7.2* [Online] Available:
<https://reference.opcfoundation.org/Core/Part14/v104/docs/>
- [12] *“Use containers to Build, Share and Run your applications”* [Online]
Available: <https://www.docker.com/resources/what-container/>
- [13] *“What is Kubernetes”* [Online] Available:
<https://kubernetes.io/docs/concepts/overview/>
- [14] *“Precision Clock Synchronization”* [Online] Available: <http://bit.ly/4qQbR49>
- [15] *“Real-Time scheduling: from hard to soft real-time systems”* [Online]
Available: https://www.researchgate.net/publication/286301706_Real-Time_scheduling_from_hard_to_soft_real-time_systems
- [16] *“Time series database explained”* [Online] Available:
<https://www.influxdata.com/time-series-database/>